

花栖小筑

BloomHaven (英文名) • V3.1.0

Source code excerpt (first 30 + last 30 pages)

Scope: Assets_Game own source. Files: 45 / Lines: 10580 / 50 lines per page / Total pages: 212

Generated: 2026-09-25 19:39:41 (local build chain, headless Edge print)

```
// ===== FILE: Assets\_Game\Scripts\Editor\AudioCheck.cs =====
using System;
using System.Collections.Generic;
using System.IO;
using System.Text;
using Sy.BloomHaven.Audio;
using Sy.BloomHaven.Garden;
using Sy.BloomHaven.Localize;
using Sy.BloomHaven.Orders;
using UnityEditor;
using UnityEngine;

namespace Sy.BloomHaven
{
    /// <summary>
    /// T008 audio wiring + copy externalization gate (batchmode entry):
    /// Tuanjie.exe -batchmode -quit -executeMethod Sy.BloomHaven.AudioCheck.Run -logFile &lt;abs>>
    /// Assert groups: A copy law (table loaded, every consumed key resolves,
    ///                 banned literal scan over Runtime sources, format keys,
    ///                 frozen-roster name-key parity)
    ///                 B stem law (four g19_stems, unknown/empty lookups null-safe)
    ///                 C hook wiring law (trio + BGM counters advance with the
    ///                 catalog empty - silent pending face, no exception)
    ///                 D catalog build law (setup tool run, asset exists on
    ///                 disk, entries match the clips actually on disk)
    /// Exit codes: 0=all groups pass 1=any assert failure or exception
    /// </summary>
    public static class AudioCheck
    {
        sealed class Row
        {
            public string Name;
            public bool Ok;
            public string Detail;
        }

        static readonly List<Row> Rows = new List<Row>();

        static void Check(string name, bool ok, string detail)
        {
            Rows.Add(new Row { Name = name, Ok = ok, Detail = detail });
        }

        // Every key the views consume (drift face: a renamed key in code but
        // not in cfg_i18n.json must fail here, never silently fall back).
        static readonly string[] ConsumedKeys =
        {
            "garden.hint", "garden.wallet", "garden.share", "garden.renovated",
            "garden.bloom_caption", "garden.payout", "garden.welcome_back", "garden.welcome_max",
            "order.title", "order.hint", "order.reward", "order.ready",
        }
    }
}
```

```

"order.accepted", "order.offered", "order.target.color", "order.target.species",
"order.target.dual", "color.blue", "color.yellow", "color.red", "color.purple",
"share.title", "share.code_label", "share.visit_label", "share.placeholder",
"share.invalid_code", "share.visited", "share.copied",
"guide.bloom_welcome", "guide.plant_prompt", "guide.order_callout",
"species_SP01", "species_SP02", "species_SP03", "species_SP04",
"order_ORD01", "order_ORD02", "order_ORD03", "order_ORD04",
"order_ORD05", "order_ORD06", "order_ORD07", "order_ORD08",
"order_ORD09", "order_ORD10", "order_ORD11", "order_ORD12",
"garden.codex", "codex.title", "codex.close", "codex.locked", "codex.locked_hint",
"codex.blooms", "codex.boosts_title", "codex.boost_speedup", "codex.boost_double",
"codex.boost_refresh", "codex.boost_speedup_done", "codex.boost_double_done",
"codex.boost_refresh_done", "codex.boost_cap_full", "codex.boost_none_growing",
"codex.boosts_resting", "floriography_SP01", "floriography_SP02",
"floriography_SP03", "floriography_SP04",
"lore_SP01", "lore_SP02", "lore_SP03", "lore_SP04",
};

// User-visible copy that must NO LONGER live as a quoted literal in
// any Runtime source (all of it moved behind cfg_i18n keys).
static readonly string[] BannedLiterals =
{
    "Renovated!", "Petals: ", "Tap an empty patch to plant - tap a bloom to harvest",
    "Welcome back! +", " petals while away", " (max)",
    "Share", "Share your garden", "Your garden code (send this to a friend)",
    "Enter a friend's code to visit their garden", "Paste a friend's code here",
    "Welcome to your friend's garden!", "Copied! Send it to a friend",
    "Code not recognized - check with your friend",
    "Renovation Orders", "Complete an order to renovate a garden block",
    "Ready to complete!", "Accepted", "Offered", " blooms", "Grow ", "Mix ", " petals",
    "First bloom! Flowers open as you watch",
    "Tap the glowing patch to plant your first flower",
    "First renovation order arrived - complete it to flip a block",
    "Codex", "Flower Codex", "Undiscovered bloom",
    "Bloom it once to reveal its story", "Blooms: ", "Little boosts (optional)",
    "Speed up growth", "Double next harvest", "Refresh orders",
    "Growth speeded up - blooms arrive sooner", "Next harvest will pay double",
    "Fresh orders rolled in", "Boosts are resting now - the garden keeps blooming",
    "Nothing is growing right now - plant a seed first", "Boosts return after a little rest",
    "Bluebell whispers: constancy and quiet gratitude",
    "Sunburst says: a small joy shared is joy doubled",
    "Roseheart holds: patience turns ruins into romance",
    "Lavendermist hums: calm outlasts every storm",
    "Old tale: bluebells ring only for promises kept",
    "Village lore: sunbursts open where children laugh",
    "A roseheart once bloomed from a fallen garden wall",
    "Travelers sewed lavendermist into hats to stay calm",
};

public static void Run()

```

```

{
    int code;
    try
    {
        Rows.Clear();
        GroupA();
        GroupB();
        GroupC();
        GroupD();
        code = 0;
        for (int i = 0; i < Rows.Count; i++)
            if (!Rows[i].Ok) { code = 1; break; }
    }
    catch (Exception e)
    {
        Debug.LogError("[AudioCheck] exception: " + e);
        Check("exception", false, e.GetType().Name + ": " + e.Message);
        code = 1;
    }

    var sb = new StringBuilder();
    int fails = 0;
    for (int i = 0; i < Rows.Count; i++)
    {
        if (!Rows[i].Ok) fails++;
        string line = "[AUDIO] " + (i + 1) + "/" + Rows.Count + " " +
            (Rows[i].Ok ? "ok" : "FAIL ") + Rows[i].Name + " | " + Rows[i].Detail;
        sb.AppendLine(line);
        Debug.Log(line);
    }
    sb.AppendLine("[AUDIO] RESULT " + (fails == 0 ? "PASS" : "FAIL") + " fails=" + fails);

    try
    {
        string logsDir = Path.Combine(Path.GetFullPath(Path.Combine(Application.dataPath, "..")), "Logs");
        if (!Directory.Exists(logsDir)) Directory.CreateDirectory(logsDir);
        File.WriteAllText(Path.Combine(logsDir, "audio_check_report.txt"),
            sb.ToString(), new UTF8Encoding(false));
    }
    catch (Exception e)
    {
        Debug.LogError("[AudioCheck] report write failed: " + e);
    }
    EditorApplication.Exit(code);
}

// A. copy law: the shipped table is live, every consumed key resolves,
// no banned literal survived in Runtime sources, format strings work.
static void GroupA()
{

```

```

bool loaded = LocalizeService.Loaded;
Check("A1 i18n table loaded", loaded,
    "count=" + (LocalizeService.All() != null ? LocalizeService.All().Count : 0));

int missing = 0;
var firstMissing = string.Empty;
var all = LocalizeService.All();
for (int i = 0; i < ConsumedKeys.Length; i++)
{
    if (all == null || !all.ContainsKey(ConsumedKeys[i]))
    {
        missing++;
        if (string.IsNullOrEmpty(firstMissing)) firstMissing = ConsumedKeys[i];
    }
}
Check("A2 consumed keys resolve", missing == 0,
    missing == 0 ? "keys=" + ConsumedKeys.Length : "missing=" + missing + " first=" + firstMissing);

string runtimeDir = Path.Combine(Application.dataPath, "_Game/Scripts/Runtime");
string[] sources = Directory.GetFiles(runtimeDir, "*.cs", SearchOption.AllDirectories);
int hits = 0;
string hitFile = string.Empty;
string hitLiteral = string.Empty;
foreach (string file in sources)
{
    string text = File.ReadAllText(file);
    foreach (string literal in BannedLiterals)
    {
        if (text.Contains "\"" + literal + "\"")
        {
            hits++;
            if (hits == 1)
            {
                hitFile = Path.GetFileName(file);
                hitLiteral = literal;
            }
        }
    }
}
Check("A3 zero hardcoded copy scan", hits == 0,
    hits == 0 ? "sources=" + sources.Length + " literals=" + BannedLiterals.Length
        : "hits=" + hits + " first=" + hitFile + ":" + hitLiteral);

bool walletOk = string.Equals(LocalizeService.GetF("garden.wallet", 12), "Petals: 12", StringComparison.Ordinal);
Check("A4 format keys", walletOk
    && string.Equals(LocalizeService.GetF("order.reward", 5), "+5 petals", StringComparison.Ordinal)
    && string.Equals(LocalizeService.GetF("garden.bloom_caption", "bluebell", 1), "bluebell +1",
StringComparison.Ordinal)
    && string.Equals(LocalizeService.GetF("codex.blooms", 3), "Blooms: 3", StringComparison.Ordinal),
    "wallet=[" + LocalizeService.GetF("garden.wallet", 12) + "]);

```

```

int nameKeyBad = 0;
for (int i = 0; i < SpeciesTable.Count; i++)
{
    var def = SpeciesTable.Get(i);
    if (def == null || !all.ContainsKey(def.NameKey)
        || !string.Equals(LocalizeService.Get(def.NameKey), def.NameEn, StringComparison.Ordinal))
        nameKeyBad++;
}
for (int i = 0; i < OrderTable.PoolCount; i++)
{
    var def = OrderTable.Get(i);
    if (def == null || !all.ContainsKey(def.NameKey)
        || !string.Equals(LocalizeService.Get(def.NameKey), def.NameEn, StringComparison.Ordinal))
        nameKeyBad++;
}
Check("A5 roster name-key parity", nameKeyBad == 0,
    nameKeyBad == 0 ? "species=4 orders=12" : "bad=" + nameKeyBad);
}

// B. stem law: the four stems exist, follow the g19_naming law and
// resolve null-safe (no catalog or unknown key never throws).
static void GroupB()
{
    bool four = AudioStems.All.Length == 4;
    bool prefixed = true;
    bool unique = true;
    for (int i = 0; i < AudioStems.All.Length; i++)
    {
        if (!AudioStems.All[i].StartsWith("g19_", StringComparison.Ordinal)) prefixed = false;
        for (int j = i + 1; j < AudioStems.All.Length; j++)
            if (string.Equals(AudioStems.All[i], AudioStems.All[j], StringComparison.Ordinal)) unique = false;
    }
    Check("B1 stem set law", four && prefixed && unique,
        "n=" + AudioStems.All.Length + " prefixed=" + prefixed + " unique=" + unique);

    bool noThrowUnknown = false;
    try
    {
        AudioClip unknown = G19AudioRuntime.Get("g19_sfx_nope");
        noThrowUnknown = unknown == null;
    }
    catch (Exception) { noThrowUnknown = false; }
    bool noThrowEmpty = false;
    try
    {
        AudioClip empty = G19AudioRuntime.Get(string.Empty);
        noThrowEmpty = empty == null;
    }
    catch (Exception) { noThrowEmpty = false; }
    Check("B2 lookup null-safe", noThrowUnknown && noThrowEmpty,

```

```

        "unknown=" + noThrowUnknown + " empty=" + noThrowEmpty));
    }

    // C. hook wiring law: every entry point fires with the catalog in its
    // current state (clips pending = counters still advance, zero throw,
    // BGM face stays honest).
    static void GroupC()
    {
        AudioHost host = AudioHost.EnsureHost();
        bool created = host != null && host.gameObject.name == AudioHost.HostName;
        Check("C1 host created", created, "host=" + (host != null ? host.gameObject.name : "null"));

        AudioHost.PlayBloomHook();
        AudioHost.PlayPlantThud();
        AudioHost.PlayOrderChime();
        AudioHost.PlayOrderRefresh();
        AudioHost.StartBgm();
        bool counters = host.BloomHookFired >= 1 && host.PlantHookFired >= 1
            && host.OrderChimeFired >= 1 && host.OrderRefreshFired >= 1 && host.BgmStartCalls >= 1;
        Check("C2 hook counters (trio+refresh)", counters,
            "bloom=" + host.BloomHookFired + " plant=" + host.PlantHookFired
            + " order=" + host.OrderChimeFired + " refresh=" + host.OrderRefreshFired
            + " bgm=" + host.BgmStartCalls);

        // Asset-state-agnostic honest mirror: the BGM face must always
        // report exactly whether a clip resolved (pending now, true later).
        bool bgmMirror = host.BgmClipPresent == (G19AudioRuntime.Get(AudioStems.BgmGarden) != null);
        int pending = 0;
        for (int i = 0; i < AudioStems.All.Length; i++)
            if (G19AudioRuntime.Get(AudioStems.All[i]) == null) pending++;
        Check("C3 silent pending face", bgmMirror,
            "bgmClipPresent=" + host.BgmClipPresent + " stemsPending=" + pending + "/" + AudioStems.All.Length);
    }

    // D. catalog build law: run the setup tool, then the asset exists on
    // disk and its entries mirror the clips actually under Art/Audio.
    static void GroupD()
    {
        int built = G19AudioSetup.BuildCatalog();
        G19AudioRuntime.ResetCache();
        bool exists = File.Exists("Assets/_Game/Resources/G19AudioCatalog.asset");
        Check("D1 catalog asset built", exists, "entries=" + built);

        int diskClips = 0;
        foreach (string guid in AssetDatabase.FindAssets("t:AudioClip", new[] { "Assets/_Game/Art/Audio" }))
            diskClips++;
        var catalog = G19AudioRuntime.Catalog;
        bool mirror = catalog != null && catalog.entries.Count == diskClips;
        Check("D2 catalog-disk mirror", mirror,
            "catalog=" + (catalog != null ? catalog.entries.Count : -1) + " disk=" + diskClips);
    }

```

```

        bool report = File.Exists("Logs/g19_audio_catalog_report.txt");
        Check("D3 setup report written", report, "path=Logs/g19_audio_catalog_report.txt");
    }
}
}
// ===== FILE: Assets\\_Game\\Scripts\\Editor\\BootSequenceCheck.cs =====
using System.IO;
using System.Text;
using UnityEditor;
using UnityEditor.SceneManagement;
using UnityEngine;

namespace Sy.BloomHaven
{
    /// <summary>
    /// Seven-step boot dry-run (batchmode entry):
    /// Tuanjie.exe -batchmode -quit -executeMethod Sy.BloomHaven.BootSequenceCheck.Run -logFile <abs>
    /// Exit codes: 0=all steps pass and well formed 1=fail/exception 3=scenes missing (run ProjectSetup first)
    /// </summary>
    public static class BootSequenceCheck
    {
        public static void Run()
        {
            int code;
            try
            {
                string mainAsset = "Assets/_Game/Scenes/S19_Main.unity";
                string projectRoot = Path.GetFullPath(Path.Combine(Application.dataPath, ".."));
                if (!File.Exists(Path.Combine(projectRoot, mainAsset)))
                {
                    Debug.LogError("[BootSequenceCheck] main scene missing - run ProjectSetup first");
                    EditorApplication.Exit(3);
                    return;
                }

                Boot.BootServices.RegisterAll();
                bool switched = false;
                var sequence = new Boot.BootSequence(delegate
                {
                    EditorSceneManager.OpenScene(mainAsset, OpenSceneMode.Single);
                    switched = true;
                    return true;
                });
                bool allOk = sequence.RunAll();

                var sb = new StringBuilder();
                foreach (var line in sequence.Lines) sb.AppendLine(line);
                var logsDir = Path.Combine(projectRoot, "Logs");
                if (!Directory.Exists(logsDir)) Directory.CreateDirectory(logsDir);
            }
            catch { }
        }
    }
}

```



```

        File.WriteAllText(Path.Combine(logsDir, "boot_sequence_report.txt"), sb.ToString(), new UTF8Encoding(false));

        bool wellFormed = sequence.Lines.Count == Boot.BootSequence.TotalSteps && switched;
        for (int i = 0; i < sequence.Lines.Count; i++)
        {
            Debug.Log(sequence.Lines[i]);
            if (!sequence.Lines[i].StartsWith("[BOOT] " + (i + 1) + "/" + Boot.BootSequence.TotalSteps + " "))
                wellFormed = false;
        }

        code = allOk && wellFormed ? 0 : 1;
    }
    catch (System.Exception e)
    {
        Debug.LogError("[BootSequenceCheck] exception: " + e);
        code = 1;
    }
    EditorApplication.Exit(code);
}
}

// ===== FILE: Assets\_Game\Scripts\Editor\BuildService.cs =====
using System;
using System.Collections.Generic;
using System.IO;
using System.Reflection;
using System.Text;
using UnityEditor;
using UnityEditor.Build;
using UnityEngine;

namespace Sy.BloomHaven
{
    /// <summary>
    /// Compile zero-warning gate stub (batchmode entry).
    /// Usage: Tuanjie.exe -batchmode -quit -projectPath <p> -executeMethod Sy.BloomHaven.BuildService.CompileCheck -logFile <abs>
    /// Exit codes: 0=zero error zero warning 1=has error 2=has warning 3=no -logFile 4=exception
    /// </summary>
    public static class BuildService
    {
        public const int Ok = 0;
        public const int HasError = 1;
        public const int HasWarning = 2;
        public const int NoLogFile = 3;
        public const int Exception = 4;

        public static void CompileCheck()
        {
            var report = new CompileCheckReport
            {

```

```

        project = "G19_BloomHaven",
        unityVersion = Application.unityVersion,
        atUtc = DateTime.UtcNow.ToString("o"),
        errors = new List<string>(),
        warnings = new List<string>(),
    };

    int code = Ok;
    try
    {
        string logPath = FindArgValue("-logFile");
        if (string.IsNullOrEmpty(logPath) || !File.Exists(logPath))
        {
            report.status = "no_log_file";
            code = NoLogFile;
        }
        else
        {
            ScanLog(logPath, report);
            if (report.errors.Count > 0) { report.status = "error"; code = HasError; }
            else if (report.warnings.Count > 0) { report.status = "warning"; code = HasWarning; }
            else { report.status = "ok"; code = Ok; }
        }
    }
    catch (Exception e)
    {
        report.status = "exception";
        report.exception = e.Message + "\n" + e.StackTrace;
        code = Exception;
    }

    WriteReport(report);
    EditorApplication.Exit(code);
}

/// <summary>Reads a command line value, supporting both "-logFile x" and "-logFile=x".</summary>
internal static string FindArgValue(string name)
{
    var args = Environment.GetCommandLineArgs();
    for (int i = 0; i < args.Length; i++)
    {
        if (string.Equals(args[i], name, StringComparison.OrdinalIgnoreCase))
        {
            if (i + 1 < args.Length) return args[i + 1];
        }
        else if (args[i].StartsWith(name + "=", StringComparison.OrdinalIgnoreCase))
        {
            return args[i].Substring(name.Length + 1);
        }
    }
}

```

```

        return null;
    }

    /// <summary>Counts compile errors/warnings (in batchmode compilation happens before executeMethod).</summary>
    static void ScanLog(string logPath, CompileCheckReport report)
    {
        using var fs = new FileStream(logPath, FileMode.Open, FileAccess.Read, FileShare.ReadWrite);
        using var sr = new StreamReader(fs, Encoding.UTF8);
        var seenErrors = new HashSet<string>();
        var seenWarnings = new HashSet<string>();
        string line;
        while ((line = sr.ReadLine()) != null)
        {
            if (line.IndexOf(": error CS", StringComparison.Ordinal) >= 0
                || line.IndexOf("Scripts have compiler errors", StringComparison.Ordinal) >= 0
                || line.IndexOf("Error building Player", StringComparison.Ordinal) >= 0)
            {
                if (seenErrors.Add(Trim(line))) report.errors.Add(Trim(line));
            }
            else if (line.IndexOf(": warning CS", StringComparison.Ordinal) >= 0)
            {
                if (seenWarnings.Add(Trim(line))) report.warnings.Add(Trim(line));
            }
        }
        report.errorCount = report.errors.Count;
        report.warningCount = report.warnings.Count;
    }

    static string Trim(string s)
    {
        s = s.TrimEnd();
        return s.Length > 500 ? s.Substring(0, 500) + "... " : s;
    }

    static void WriteReport(CompileCheckReport report)
    {
        try
        {
            var projectRoot = Path.GetFullPath(Path.Combine(Application.dataPath, ".."));
            var logsDir = Path.Combine(projectRoot, "Logs");
            if (!Directory.Exists(logsDir)) Directory.CreateDirectory(logsDir);
            File.WriteAllText(
                Path.Combine(logsDir, "compile_report.json"),
                JsonUtility.ToJson(report, true),
                new UTF8Encoding(false));
            Console.WriteLine($"[CompileCheck] status={report.status} errors={report.errorCount} warnings={report.warningCount} -
> Logs/compile_report.json");
        }
        catch (Exception e)
        {
            Console.WriteLine("[CompileCheck] report write failed: " + e.Message);
        }
    }

```

```

    }
}

/// <summary>
/// S1Gate closeout: real weixin-minigame build entry (batchmode). Ported
/// from Sy.CrazyEstate.BuildService.BuildWeixin (G16 paradigm; G15
/// PetWorkCrew origin, P02/P06 proven lineage): reads pinned
/// EditorBuildSettings scenes, switches to the WeixinMiniGame target,
/// pins the raw-container compression format (X123: brotli output would
/// break the later 4MB main-package split contract), builds with the
/// SY_WECHAT define pinned (X129 single-define law), and writes the
/// machine-readable logs/build_weixin_report.json. The fmt pin is
/// restored in finally. Douyin stays capability-gated in
/// tools/BuildPackage.ps1 (no starksdk face vendored yet = NA verdict).
/// Exit 0 = build succeeded, 1 = failure, 4 = exception.
/// Usage: Tuanjie.exe -batchmode -quit -projectPath &lt;p>&gt; -executeMethod Sy.BloomHaven.BuildService.BuildWeixin -logFile
&lt;abs&gt;
/// </summary>
public static void BuildWeixin()
{
    int code;
    SerializedObject psObj = null;
    int fmtPrev = 0;
    bool pinned = false;
    try
    {
        var scenes = EditorBuildSettings.scenes;
        if (scenes == null || scenes.Length == 0)
        {
            Console.WriteLine("[BuildWeixin] no scenes pinned in EditorBuildSettings");
            EditorApplication.Exit(1);
            return;
        }
        var levellist = new List<string>();
        foreach (var s in scenes)
            if (s.enabled) levellist.Add(s.path);
        if (levellist.Count == 0)
        {
            Console.WriteLine("[BuildWeixin] no enabled scenes");
            EditorApplication.Exit(1);
            return;
        }

        bool switched = EditorUserBuildSettings.SwitchActiveBuildTarget(
            BuildTargetGroup.WeixinMiniGame, BuildTarget.WeixinMiniGame);
        Console.WriteLine("[BuildWeixin] switchActiveTarget=" + switched
            + " scenes=" + levellist.Count);
        if (!switched) { EditorApplication.Exit(1); return; }

        string projectRoot = Path.GetFullPath(Path.Combine(Application.dataPath, ".."));
        string outDir = Path.Combine(projectRoot, "MiniGame", "weixin", "build");
    }
    catch { }
}

```

```

Directory.CreateDirectory(outDir);

// X123: pin the raw-container format before building. Abort before
// BuildPlayer when the key cannot be driven.
var psAll = UnityEngine.Resources.FindObjectsOfTypeAll(typeof(PlayerSettings));
if (psAll == null || psAll.Length == 0)
{
    Console.WriteLine("[BuildWeixin] X123 abort: PlayerSettings instance not found");
    EditorApplication.Exit(1);
    return;
}
psObj = new SerializedObject(psAll[0]);
var fmtProp = psObj.FindProperty("weixinMiniGameCompressionFormat");
if (fmtProp == null)
{
    Console.WriteLine("[BuildWeixin] X123 abort: serialized key weixinMiniGameCompressionFormat missing");
    EditorApplication.Exit(1);
    return;
}
fmtPrev = fmtProp.intValue;
fmtProp.intValue = 0;
psObj.ApplyModifiedProperties();
Console.WriteLine("[BuildWeixin] X123 fmtPin prev=" + fmtPrev
    + " set=0 readback=" + fmtProp.intValue);
if (fmtProp.intValue != 0)
{
    Console.WriteLine("[BuildWeixin] X123 abort: pin readback mismatch - no build");
    EditorApplication.Exit(1);
    return;
}
pinned = true;

string[] buildDefines = { "SY_WECHAT" };

var report = BuildPipeline.BuildPlayer(new BuildPlayerOptions
{
    scenes = levelList.ToArray(),
    locationPathName = outDir,
    target = BuildTarget.WeixinMiniGame,
    options = BuildOptions.None,
    extraScriptingDefines = buildDefines,
});

string result = report.summary.result.ToString();
long totalBytes = (long)report.summary.totalSize;
Console.WriteLine("[BuildWeixin] result=" + result
    + " totalSizeBytes=" + totalBytes
    + " errors=" + report.summary.totalErrors
    + " warnings=" + report.summary.totalWarnings);

```

```

try
{
    var logsDir = Path.Combine(projectRoot, "Logs");
    if (!Directory.Exists(logsDir)) Directory.CreateDirectory(logsDir);
    File.WriteAllText(
        Path.Combine(logsDir, "build_weixin_report.json"),
        "{\result\": \"\" + result + "\",\totalSizeBytes\": \"\" + totalBytes
        + "\",\errors\": \"\" + report.summary.totalErrors
        + "\",\warnings\": \"\" + report.summary.totalWarnings
        + "\",\defineInjected\": \"\" + string.Join(\";\", buildDefines) + \"\"
        + "\",\atUtc\": \"\" + DateTime.UtcNow.ToString(\"o\") + \"\"}",
        new UTF8Encoding(false));
    }
    catch (Exception e) { Console.WriteLine("[BuildWeixin] report write failed: " + e.Message); }

    code = report.summary.result == UnityEditor.Build.Reporting.BuildResult.Succeeded ? 0 : 1;
}
catch (Exception e)
{
    Console.WriteLine("[BuildWeixin] exception: " + e.Message + " | " + e.StackTrace);
    code = Exception;
}
finally
{
    if (pinned && psObj != null)
    {
        try
        {
            psObj.Update();
            var rp = psObj.FindProperty("weixinMiniGameCompressionFormat");
            if (rp != null)
            {
                rp.intValue = fmtPrev;
                psObj.ApplyModifiedProperties();
                Console.WriteLine("[BuildWeixin] X123 fmtRestored=" + rp.intValue);
            }
            else
            {
                Console.WriteLine("[BuildWeixin] X123 WARNING: restore key missing");
            }
        }
        catch (Exception rex)
        {
            Console.WriteLine("[BuildWeixin] X123 restore failed: " + rex.Message);
        }
    }
}
EditorApplication.Exit(code);
}

```

```

// 鉞鉞 Douyin build face (MiniGame subplatform system, X902 port) 鉞鉞
// Ported from Sy.P08.BuildService.BuildDouyin (P08 douyin success
// precedent, X1033 same-shape vendored starksdk; G16/X902 law,
// P02/P06 X121/X136 lineage): the douyin build rides the MiniGame
// platform (BuildTarget.MiniGame) with a subplatform selection, not
// a separate module; the stark editor assemblies do not sit in a
// fresh batchmode domain, so the sdk package dlls load explicitly
// via Assembly.LoadFrom and the carrier DouyinBuilder.BuildWebGL(
// StarkBuilderSettings) is reached through reflection. Every miss
// lands in Logs/build_douyin_report.json (honest failure surface).
// Only SY_DOUYIN is pinned transiently on the MiniGame named target
// (snapshot -> inject -> finally restore); the appId stays a
// placeholder until the console id lands at submission (user
// physical step).

internal static UnityEditor.MiniGameBuildSubtarget ResolveDouyinSubtarget()
{
    foreach (UnityEditor.MiniGameBuildSubtarget v in Enum.GetValues(typeof(UnityEditor.MiniGameBuildSubtarget)))
    {
        if (v.ToString().IndexOf("DouYin", StringComparison.OrdinalIgnoreCase) >= 0) return v;
    }
    throw new InvalidOperationException("MiniGameBuildSubtarget carries no DouYin member");
}

/// <summary>
/// Douyin build entry (batchmode). Creates the douyin build profile
/// (BuildTarget.MiniGame + douyin subtarget), auto-installs the
/// subplatform package when missing, then drives the X121-proven
/// carrier DouyinBuilder.BuildWebGL(StarkBuilderSettings) with the
/// fixed douyin profile while SY_DOUYIN is pinned on the MiniGame
/// named target (snapshot -> inject -> finally restore). Writes
/// Logs/build_douyin_report.json (status/subtarget/installed/verdict/
/// defineInjected/packageDir/atUtc). Exit 0 = package produced,
/// 1 = face miss or no package, 4 = exception.
/// Usage: Tuanjie.exe -batchmode -quit -projectPath &lt;p>&gt; -executeMethod Sy.BloomHaven.BuildService.BuildDouyin -logFile
&lt;abs&gt;
/// </summary>
public static void BuildDouyin()
{
    int code;
    string projectRoot = Path.GetFullPath(Path.Combine(Application.dataPath, ".."));
    string definesPrev = null;
    bool definePinned = false;
    var defineTarget = NamedBuildTarget.FromBuildTargetGroup(BuildTargetGroup.MiniGame);
    try
    {
        var subtarget = ResolveDouyinSubtarget();
        bool installed = UnityEditor.Build.Profile.BuildProfileSubplatformProxy.IsSubplatformInstalled(subtarget);
        if (!installed)
            UnityEditor.Build.Profile.BuildProfileSubplatformProxy.InstallSubplatform(subtarget);
        installed = UnityEditor.Build.Profile.BuildProfileSubplatformProxy.IsSubplatformInstalled(subtarget);
    }
}

```

```

var profile = UnityEditor.Build.Profile.BuildProfile.CreateInstance(
    BuildTarget.MiniGame, subtarget);
if (profile == null)
{
    Console.WriteLine("[BuildDouyin] profile creation returned null");
    WriteDouyinReport(projectRoot, "no_profile", subtarget.ToString(), installed, string.Empty);
    EditorApplication.Exit(1);
    return;
}
string outDir = Path.Combine(projectRoot, "MiniGame", "douyin", "build");
Directory.CreateDirectory(outDir);
profile.buildPath = outDir;

// X136 law (P02 lineage): the helper.Build path is dead. The
// proven carrier is DouyinBuilder.BuildWebGL(
// StarkBuilderSettings) with the fixed douyin profile
// (placeholder appId, dev build, new build format,
// web-facing publish type, real output dir).
var settingsType = FindLoadedType("TTSDK.Tool.StarkBuilderSettings");
var douyinBuilderType = FindLoadedType("TTSDK.Tool.DouyinBuilder");
if (settingsType == null || douyinBuilderType == null)
{
    Console.WriteLine("[BuildDouyin] stark sdk types missing: settings="
        + (settingsType == null) + " builder=" + (douyinBuilderType == null));
    WriteDouyinReport(projectRoot, "no_stark_types", subtarget.ToString(), installed, string.Empty);
    EditorApplication.Exit(1);
    return;
}
MethodInfo loadMethod = null;
foreach (var m in settingsType.GetMethods(BindingFlags.Public | BindingFlags.NonPublic | BindingFlags.Static))
{
    var lp = m.GetParameters();
    if (m.Name == "LoadSettings" && lp.Length == 1 && lp[0].ParameterType == typeof(bool))
    { loadMethod = m; break; }
}
if (loadMethod == null)
{
    Console.WriteLine("[BuildDouyin] StarkBuilderSettings.LoadSettings(bool) not found");
    WriteDouyinReport(projectRoot, "no_settings_load", subtarget.ToString(), installed, string.Empty);
    EditorApplication.Exit(1);
    return;
}
object settings = loadMethod.Invoke(null, new object[] { true });
if (settings == null)
{
    Console.WriteLine("[BuildDouyin] LoadSettings(true) returned null");
    WriteDouyinReport(projectRoot, "settings_load_failed", subtarget.ToString(), installed, string.Empty);
    EditorApplication.Exit(1);
    return;
}

```



```

    }
    ApplyDouyinBuildProfile(settingsType, settings, outDir);

    // The douyin helper path carries no extraScriptingDefines, so
    // SY_DOUYIN is pinned on the MiniGame named target
    // transiently (snapshot, mutate, restore in finally - the
    // editor symbol table stays SY_* free between runs).
    definesPrev = PlayerSettings.GetScriptingDefineSymbols(defineTarget);
    var defineList = new List<string>();
    foreach (var d in definesPrev.Split(';'))
    {
        string dt = d.Trim();
        if (dt.Length > 0 && !defineList.Contains(dt)) defineList.Add(dt);
    }
    if (!defineList.Contains("SY_DOUYIN")) defineList.Add("SY_DOUYIN");
    string definesMerged = string.Join(";", defineList);
    PlayerSettings.SetScriptingDefineSymbols(defineTarget, definesMerged);
    definePinned = true;
    Console.WriteLine("[BuildDouyin] definePin prev='" + definesPrev + "' set='" + definesMerged + "'");

    MethodInfo buildFace = null;
    foreach (var m in douyinBuilderType.GetMethods(BindingFlags.Public | BindingFlags.NonPublic | BindingFlags.Static))
    {
        var mp = m.GetParameters();
        if (m.Name == "BuildWebGL" && mp.Length == 1 && mp[0].ParameterType.Name == "StarkBuilderSettings")
        { buildFace = m; break; }
    }
    if (buildFace == null)
    {
        Console.WriteLine("[BuildDouyin] DouyinBuilder.BuildWebGL(StarkBuilderSettings) not found");
        WriteDouyinReport(projectRoot, "no_build_face", subtarget.ToString(), installed, string.Empty);
        EditorApplication.Exit(1);
        return;
    }

    var watch = System.Diagnostics.Stopwatch.StartNew();
    var buildTask = buildFace.Invoke(null, new object[] { settings }) as System.Threading.Tasks.Task;
    while (buildTask != null && !buildTask.IsCompleted && watch.Elapsed.TotalSeconds < 900.0)
    {
        System.Threading.Thread.Sleep(2000);
    }
    string verdict = buildTask == null ? "no_task"
        : (buildTask.IsCompleted ? buildTask.Status.ToString() : "pump_timeout");
    if (buildTask != null && buildTask.Exception != null)
        verdict = buildTask.Status + "+" + buildTask.Exception.GetBaseException().GetType().Name;
    string packageDir = FindDouyinPackageDir(outDir);
    Console.WriteLine("[BuildDouyin] subtarget=" + subtarget
        + " installed=" + installed
        + " profileType=" + profile.GetType().Name
        + " taskSeconds=" + watch.Elapsed.TotalSeconds.ToString("F1")

```

```

        + " verdict=" + verdict
        + " packageDir=" + (packageDir ?? "none"));

    WriteDouyinReport(projectRoot,
        packageDir != null ? "succeeded" : "failed",
        subtarget.ToString(), installed, verdict, "SY_DOUYIN", packageDir);
    code = packageDir != null ? Ok : 1;
}
catch (Exception e)
{
    Console.WriteLine("[BuildDouyin] exception: " + e.Message + " | " + e.StackTrace);
    WriteDouyinReport(projectRoot, "exception", string.Empty, false, e.Message);
    code = Exception;
}
finally
{
    if (definePinned)
    {
        try
        {
            PlayerSettings.SetScriptingDefineSymbols(defineTarget, definesPrev);
            Console.WriteLine("[BuildDouyin] defineRestored="
                + PlayerSettings.GetScriptingDefineSymbols(defineTarget) + "");
        }
        catch (Exception rex)
        {
            Console.WriteLine("[BuildDouyin] define restore failed: " + rex.Message);
        }
    }
}
EditorApplication.Exit(code);
}

static void WriteDouyinReport(string projectRoot, string status, string subtarget, bool installed, string verdict,
    string definesInjected = null, string packageDir = null)
{
    try
    {
        var logsDir = Path.Combine(projectRoot, "Logs");
        if (!Directory.Exists(logsDir)) Directory.CreateDirectory(logsDir);
        File.WriteAllText(
            Path.Combine(logsDir, "build_douyin_report.json"),
            "{\n\"status\": \"\" + status + \"\", \"subtarget\": \"\" + subtarget + \"\"\n"
            + ", \"subplatformInstalled\": \"\" + (installed ? \"true\" : \"false\")\n"
            + ", \"verdict\": \"\" + (verdict ?? string.Empty).Replace(\"\\\", \"'\") + \"\"\n"
            + (string.IsNullOrEmpty(definesInjected) ? string.Empty : ", \"defineInjected\": \"\" + definesInjected + \"\"\n")
            + (string.IsNullOrEmpty(packageDir) ? string.Empty : ", \"packageDir\": \"\" + packageDir.Replace(\"\\\", \"/\") + \"\"\n")
            + ", \"atUtc\": \"\" + DateTime.UtcNow.ToString(\"o\") + \"\"}",
            new UTF8Encoding(false));
    }
}

```

```

        catch (Exception e) { Console.WriteLine("[BuildDouyin] report write failed: " + e.Message); }
    }

    static bool _starkLoadTried;
    static string _starkPkgRoot;

    // X121 loading recipe: the stark editor assemblies do not sit in a
    // fresh batchmode domain, so the sdk package dlls load explicitly
    // (name filter tools/sdk/stark/tt) with an AssemblyResolve fallback
    // before the type scan.
    static Type FindLoadedType(string fullName)
    {
        if (!_starkLoadTried) LoadStarkSdkAssemblies();
        foreach (var asm in AppDomain.CurrentDomain.GetAssemblies())
        {
            try
            {
                foreach (var t in asm.GetTypes())
                {
                    if (t.FullName == fullName) return t;
                }
            }
            catch (ReflectionTypeLoadException r)
            {
                foreach (var t in r.Types)
                {
                    if (t != null && t.FullName == fullName) return t;
                }
            }
            catch { }
        }
        return null;
    }

    static void LoadStarkSdkAssemblies()
    {
        _starkLoadTried = true;
        string projectRoot = Path.GetFullPath(Path.Combine(Application.dataPath, ".."));
        string pkgRoot = Path.Combine(projectRoot, "Packages", "com.bytedance.starksdk");
        if (!Directory.Exists(pkgRoot))
        {
            string legacy = Path.Combine(projectRoot, "LocalPackages");
            if (Directory.Exists(legacy))
            {
                foreach (var d in Directory.GetDirectories(legacy))
                {
                    if (Path.GetFileName(d).IndexOf("starksdk", StringComparison.OrdinalIgnoreCase) >= 0)
                    {
                        pkgRoot = d; break; }
                }
            }
        }
        if (!Directory.Exists(pkgRoot)) return;
        _starkPkgRoot = pkgRoot;
        try { AppDomain.CurrentDomain.AssemblyResolve += ResolveStarkFromDomain; } catch { }
        int loaded = 0;
        foreach (var dll in Directory.GetFiles(pkgRoot, "*.dll", SearchOption.AllDirectories))

```

```

    {
        string name = Path.GetFileName(dll);
        if (name.IndexOf("tools", StringComparison.OrdinalIgnoreCase) < 0
            && name.IndexOf("sdk", StringComparison.OrdinalIgnoreCase) < 0
            && name.IndexOf("stark", StringComparison.OrdinalIgnoreCase) < 0
            && name.IndexOf("tt", StringComparison.OrdinalIgnoreCase) < 0) continue;
        try { Assembly.LoadFrom(dll); loaded++; } catch { }
    }
    Console.WriteLine("[BuildDouyin] stark assemblies loaded=" + loaded + " from " + pkgRoot);
}

static Assembly ResolveStarkFromDomain(object sender, ResolveEventArgs args)
{
    string simple = (args.Name ?? "").Split(',')[0].Trim();
    foreach (var a in AppDomain.CurrentDomain.GetAssemblies())
    {
        try { if (a.GetName().Name == simple) return a; } catch { }
    }
    try
    {
        if (_starkPkgRoot != null)
        {
            foreach (var dll in Directory.GetFiles(_starkPkgRoot, simple + ".dll", SearchOption.AllDirectories))
            {
                return Assembly.LoadFrom(dll);
            }
        }
    }
    catch { }
    return null;
}

// X121-proven douyin profile: real output dir, placeholder appId
// (console ids land at submission), dev build for the device batch,
// new build format (the X120 assert root cause), and web-facing
// framework/publish enums when present.
static void ApplyDouyinBuildProfile(Type settingsType, object settings, string outputDir)
{
    SetDouyinField(settings, "OutputDir", outputDir);
    SetDouyinFieldOrProperty(settings, "_appId", "tt1111111111111111");
    SetDouyinFieldOrProperty(settings, "version", "0.1.0");
    SetDouyinField(settings, "isDevBuild", true);
    SetDouyinField(settings, "isOldBuildFormat", false);
    ResolveDouyinEnumField(settings, "framework", new[] { "WebGL", "Web", "H5" });
    ResolveDouyinEnumField(settings, "_publishType", new[] { "WebGL", "Web", "MiniGame", "Game" });
    try
    {
        foreach (var m in settingsType.GetMethods(BindingFlags.Public | BindingFlags.NonPublic | BindingFlags.Instance))
        {
            if ((m.Name == "SaveSettings" || m.Name == "Save") && m.GetParameters().Length == 0)
            { m.Invoke(settings, null); break; }
        }
    }
}

```

```

    }
    catch (Exception e) { Console.WriteLine("[BuildDouyin] settings persist failed: " + e.Message); }
    Console.WriteLine("[BuildDouyin] profile applied: OutputDir=" + outputDir);
}

static void SetDouyinField(object target, string name, object value)
{
    var f = target.GetType().GetField(name, BindingFlags.Public | BindingFlags.NonPublic | BindingFlags.Instance);
    if (f != null) f.SetValue(target, value);
}

static void SetDouyinFieldOrProperty(object target, string name, object value)
{
    var f = target.GetType().GetField(name, BindingFlags.Public | BindingFlags.NonPublic | BindingFlags.Instance);
    if (f != null) { f.SetValue(target, value); return; }
    var p = target.GetType().GetProperty(name, BindingFlags.Public | BindingFlags.NonPublic | BindingFlags.Instance);
    if (p != null && p.CanWrite) p.SetValue(target, value, null);
}

static void ResolveDouyinEnumField(object settings, string fieldName, string[] preferred)
{
    var f = settings.GetType().GetField(fieldName, BindingFlags.Public | BindingFlags.NonPublic | BindingFlags.Instance);
    if (f == null || !f.FieldType.IsEnum) return;
    string[] names = Enum.GetNames(f.FieldType);
    string chosen = null;
    foreach (var want in preferred)
    {
        foreach (var n in names)
        {
            if (string.Equals(n, want, StringComparison.OrdinalIgnoreCase)) { chosen = n; break; }
            if (chosen != null) break;
        }
        if (chosen == null)
        {
            foreach (var want in preferred)
            {
                foreach (var n in names)
                {
                    if (n.IndexOf(want, StringComparison.OrdinalIgnoreCase) >= 0) { chosen = n; break; }
                    if (chosen != null) break;
                }
            }
        }
        if (chosen != null) f.SetValue(settings, Enum.Parse(f.FieldType, chosen));
        Console.WriteLine("[BuildDouyin] enum " + fieldName + "=" + (chosen ?? "(default)"));
    }
}

static string FindDouyinPackageDir(string outputDir)
{
    if (!Directory.Exists(outputDir)) return null;
    var queue = new Queue<string>();
    queue.Enqueue(outputDir);
    while (queue.Count > 0)

```

```

        {
            string dir = queue.Dequeue();
            try
            {
                if (File.Exists(Path.Combine(dir, "game.json"))
                    && File.Exists(Path.Combine(dir, "game.js"))) return dir;
                foreach (var sub in Directory.GetDirectories(dir)) queue.Enqueue(sub);
            }
            catch { }
            if (queue.Count > 64) break;
        }
        return null;
    }
}

[Serializable]
public class CompileCheckReport
{
    public string project;
    public string unityVersion;
    public string atUtc;
    public string status;
    public int errorCount;
    public int warningCount;
    public List<string> errors;
    public List<string> warnings;
    public string exception;
}
}

// ===== FILE: Assets\_Game\Scripts\Editor\CodexIaaCheck.cs =====
using System;
using System.Collections.Generic;
using System.IO;
using System.Text;
using System.Text.RegularExpressions;
using Sy.BloomHaven.Codex;
using Sy.BloomHaven.Garden;
using Sy.BloomHaven.Iaa;
using Sy.BloomHaven.Localize;
using Sy.BloomHaven.Orders;
using Sy.BloomHaven.Save;
using Sy.BloomHaven.Share;
using UnityEditor;
using UnityEngine;

namespace Sy.BloomHaven
{
    /// <summary>
    /// T010-a codex + IAA data-layer gate (batchmode entry):
    /// A codex frozen-roster law (fresh zero, roster closure, unknown reject, completion)

```

```

/// B unlock monotonic law      (first-bloom unlock, no re-entry, count monotone,
///                               event-subscription auto-unlock via Tick = offline face)
/// C placement law + controls  (three rewarded types, unknown reject incl. interstitial,
///                               order-refresh mechanism control, no-ad completion, U149 constants)
/// D U149 rolling-cap law       (12 ok, 13th rejected, window slide restores, prune face)
/// E determinism double-run     (same script over two fresh saves -> byte-identical json)
/// F save validator healing     (null codex/adcaps lists healed on construction)
/// G U129 policy scan + control  (shipped copy face + new sources clean, scanner proven)
/// H view mechanism law         (T010-b: speedup halves the remaining window,
///                               double is one-shot and cap-clipped, refresh reroutes
///                               the board, facade wires codex+iaa live)
/// Report: Logs/g19_codex_iaa_check_report.txt. Exit codes: 0=all pass 1=any failure.
/// </summary>
public static class CodexIaaCheck
{
    // U129 banned-word dictionary (authority = G16 script sec-3b, 19 word
    // families; unicode escapes keep the source pure ASCII).
    const string PolicyWordPattern =
        "\u653F\u7B56|\u65B0\u653F|\u8C03\u63A7|\u68DA\u6539|\u65E7\u6539|\u9650\u8D2D|\u9650\u8D37|\u9650\u552E"
        + "|\u623F\u4EA7\u7A0E|\u6301\u6709\u7A0E|\u5951\u7A0E|\u79DF\u8D41\u5E76\u4E3E|\u4FDD\u4EA4\u697C"
        + "|\u623F\u4F4F\u4E0D\u7092|\u4E09\u6761\u7EA2\u7EBF|\u516C\u79EF\u91D1|\u843D\u6237|\u5F81\u6536|\u7ECF\u6D4E\u9002\u7528";

    const int Seed = 91025;

    sealed class Row { public string Name; public bool Ok; public string Detail; }

    static readonly List<Row> Rows = new List<Row>();

    static void Check(string name, bool ok, string detail)
    {
        Rows.Add(new Row { Name = name, Ok = ok, Detail = detail });
    }

    public static void Run()
    {
        int code;
        try
        {
            Rows.Clear();
            GroupA();
            GroupB();
            GroupC();
            GroupD();
            GroupE();
            GroupF();
            GroupG();
            GroupH();
            code = 0;
            for (int i = 0; i < Rows.Count; i++)
                if (!Rows[i].Ok) { code = 1; break; }
        }
    }
}

```

```

    }
    catch (Exception e)
    {
        Debug.LogError("[CodexIaaCheck] exception: " + e);
        Check("exception", false, e.GetType().Name + ": " + e.Message);
        code = 1;
    }

    var sb = new StringBuilder();
    int fails = 0;
    for (int i = 0; i < Rows.Count; i++)
    {
        if (!Rows[i].Ok) fails++;
        string line = "[CODEXIAA] " + (i + 1) + "/" + Rows.Count + " " +
            (Rows[i].Ok ? "ok" : "FAIL ") + Rows[i].Name + " | " + Rows[i].Detail;
        sb.AppendLine(line);
        Debug.Log(line);
    }
    sb.AppendLine("[CODEXIAA] RESULT " + (fails == 0 ? "PASS" : "FAIL") + " fails=" + fails);

    try
    {
        string logsDir = Path.Combine(Path.GetFullPath(Path.Combine(Application.dataPath, "..")), "Logs");
        if (!Directory.Exists(logsDir)) Directory.CreateDirectory(logsDir);
        File.WriteAllText(Path.Combine(logsDir, "codex_iaa_check_report.txt"),
            sb.ToString(), new UTF8Encoding(false));
    }
    catch (Exception e)
    {
        Debug.LogError("[CodexIaaCheck] report write failed: " + e);
    }
    EditorApplication.Exit(code);
}

// A. codex frozen-roster law: fresh face starts at zero, only the four frozen
// species can enter, and the collection completes exactly at roster size.
static void GroupA()
{
    var state = new SaveState();
    var codex = new CodexSystem(new GardenSystem(state.Garden), state);
    bool fresh = codex.UnlockedCount == 0 && !codex.IsComplete;
    Check("A1_codex_fresh_zero", fresh, "unlocked=" + codex.UnlockedCount);

    bool unknownRejected = !codex.NoteBloom("SP99", 1.0) && !codex.NoteBloom(string.Empty, 1.0)
        && codex.UnlockedCount == 0;
    Check("A2_unknown_species_rejected", unknownRejected, "unlocked=" + codex.UnlockedCount);

    codex.NoteBloom(SpeciesTable.Bluebell, 1.0);
    codex.NoteBloom(SpeciesTable.Sunburst, 2.0);
    codex.NoteBloom(SpeciesTable.Roseheart, 3.0);
}

```



```

    bool atThree = codex.UnlockedCount == 3 && !codex.IsComplete;
    codex.NoteBloom(SpeciesTable.Lavendermist, 4.0);
    bool complete = codex.UnlockedCount == SpeciesTable.Count && codex.IsComplete
        && codex.IsUnlocked(SpeciesTable.Bluebell) && codex.IsUnlocked(SpeciesTable.Lavendermist);
    Check("A3_completion_at_roster_size", atThree && complete,
        "atThree=" + atThree + " final=" + codex.UnlockedCount + "/" + SpeciesTable.Count);
}

// B. unlock monotonic law: first bloom stamps the unlock time, re-blooms never
// re-enter, bloom counts only rise, and the event subscription unlocks
// through the shared Tick path (the offline-unlock face of S5).
static void GroupB()
{
    var state = new SaveState();
    var garden = new GardenSystem(state.Garden);
    var codex = new CodexSystem(garden, state);

    bool first = codex.NoteBloom(SpeciesTable.Bluebell, 5.0);
    bool stamped = first && codex.UnlockedCount == 1 && Math.Abs(codex.Entries[0].UnlockedAt - 5.0) < 1e-9;
    Check("B1_first_bloom_unlocks", stamped, "unlockedAt=" + codex.Entries[0].UnlockedAt);

    codex.NoteBloom(SpeciesTable.Bluebell, 9.0);
    codex.NoteBloom(SpeciesTable.Bluebell, 12.0);
    bool noReentry = codex.UnlockedCount == 1 && Math.Abs(codex.Entries[0].UnlockedAt - 5.0) < 1e-9;
    bool monotone = codex.BloomCount(SpeciesTable.Bluebell) == 3
        && codex.BloomCount(SpeciesTable.Roseheart) == 0;
    Check("B2_rebloom_no_reentry_count_monotone", noReentry && monotone,
        "entries=" + codex.UnlockedCount + " blooms=" + codex.BloomCount(SpeciesTable.Bluebell));

    // Event-subscription face: bloom driven purely by the engine clock unlocks
    // the codex entry with no direct NoteBloom call (offline growth rides this).
    var state2 = new SaveState();
    var garden2 = new GardenSystem(state2.Garden);
    var codex2 = new CodexSystem(garden2, state2);
    state2.Garden.SelectedSpeciesId = SpeciesTable.Bluebell;
    garden2.Tick(0.0);
    garden2.Plant(0);
    garden2.Tick(3.0); // bluebell grow window is 3.0s
    bool autoUnlocked = codex2.UnlockedCount == 1 && codex2.IsUnlocked(SpeciesTable.Bluebell)
        && codex2.BloomCount(SpeciesTable.Bluebell) == 1;
    Check("B3_tick_bloom_auto_unlock", autoUnlocked,
        "unlocked=" + codex2.UnlockedCount + " blooms=" + codex2.BloomCount(SpeciesTable.Bluebell));
}

// C. placement law + positive controls: the three rewarded placements offer,
// unknown/interstitial types are rejected, the order-refresh mechanism face
// stays reachable, the whole loop completes with zero ad calls, and the U149
// constants stay frozen.
static void GroupC()
{

```

```

var state = new SaveState();
var iaa = new IaaSystem(state);

bool three = iaa.OfferAd(IaaSystem.AdPlantSpeedup, 10.0)
    && iaa.OfferAd(IaaSystem.AdHarvestDouble, 11.0)
    && iaa.OfferAd(IaaSystem.AdOrderRefresh, 12.0);
Check("C1_three_placements_offer", three, "capsules=" + state.AdCaps.Count);

bool unknown = !iaa.OfferAd("AD_INTERSTITIAL", 13.0) && !iaa.OfferAd(string.Empty, 13.0)
    && !iaa.OfferAd("ad_plant_speedup", 13.0) && state.AdCaps.Count == 3
    && !IaaSystem.IsPlacement("AD_INTERSTITIAL");
Check("C2_unknown_and_interstitial_rejected", unknown,
    "capsules=" + state.AdCaps.Count + " interstitialIsPlacement=false");

// Order-refresh positive control: the T004 mechanism face stays reachable
// through a rewarded offer (board re-rolls back to a full legal board).
var stateOrd = new SaveState();
var sys = new SaveSystem(stateOrd, new System.Random(Seed));
var iaaOrd = new IaaSystem(stateOrd);
sys.Orders.EnsureBoard();
bool offered = iaaOrd.OfferAd(IaaSystem.AdOrderRefresh, 50.0);
sys.Orders.RefreshBoard();
bool boardLegal = sys.Orders.State.Board.Count == OrderSystem.BoardSize;
for (int i = 0; i < sys.Orders.State.Board.Count && boardLegal; i++)
    boardLegal = OrderTable.Get(sys.Orders.State.Board[i].DefId) != null;
Check("C3_order_refresh_mechanism_control", offered && boardLegal,
    "offered=" + offered + " board=" + sys.Orders.State.Board.Count);

// No-ad completion law (U091): the full core loop closes with zero ad calls.
var stateNoAd = new SaveState();
var sysNoAd = new SaveSystem(stateNoAd, new System.Random(Seed));
sysNoAd.State.SelectedSpeciesId = SpeciesTable.Bluebell;
sysNoAd.Garden.Tick(0.0);
sysNoAd.Garden.Plant(0); sysNoAd.Garden.Plant(1); sysNoAd.Garden.Plant(2);
sysNoAd.Garden.Tick(3.0); // three live bluebells count toward order targets (no harvest yet)
sysNoAd.Orders.EnsureBoard();
int rolls = 0;
while (!sysNoAd.Orders.IsOnBoard(OrderTable.Ord01) && rolls < 12)
{ sysNoAd.Orders.RefreshBoard(); rolls++; }
var completeOrd = sysNoAd.Orders.CompleteOrder(OrderTable.Ord01); // satisfied at plant time (SG2 paradigm)
string code = ShareCodec.ComposeShare(sysNoAd.Garden.State);
bool noAd = stateNoAd.AdCaps.Count == 0 && completeOrd.Ok && completeOrd.Gained > 0
    && code.StartsWith(ShareCodec.Prefix, StringComparison.Ordinal);
Check("C4_no_ad_full_completion", noAd,
    "adcaps=" + stateNoAd.AdCaps.Count + " ok=" + completeOrd.Ok + " rolls=" + rolls
    + " gained=" + completeOrd.Gained + " wallet=" + stateNoAd.Garden.Petals);

bool frozen = IaaSystem.DailyCap == 12 && Math.Abs(IaaSystem.WindowSeconds - 86400.0) < 1e-9;
Check("C5_u149_constants_frozen", frozen,
    "cap=" + IaaSystem.DailyCap + " window=" + IaaSystem.WindowSeconds);

```

```

}

// D. U149 rolling-cap law: twelve offers land, the thirteenth is rejected, and
// the window sliding past restores room while pruning stale capsules.
static void GroupD()
{
    var state = new SaveState();
    var iaa = new IaaSystem(state);
    string[] rotate = { IaaSystem.AdPlantSpeedup, IaaSystem.AdHarvestDouble, IaaSystem.AdOrderRefresh };

    int accepted = 0;
    for (int i = 0; i < IaaSystem.DailyCap; i++)
        if (iaa.OfferAd(rotate[i % rotate.Length], 100.0 + i)) accepted++;
    bool twelve = accepted == IaaSystem.DailyCap && state.AdCaps.Count == IaaSystem.DailyCap
        && iaa.CountRecent(111.0) == IaaSystem.DailyCap;
    Check("D1_twelve_offers_accepted", twelve, "accepted=" + accepted);

    bool thirteenth = !iaa.OfferAd(IaaSystem.AdPlantSpeedup, 112.0) && !iaa.CapHasRoom(112.0)
        && state.AdCaps.Count == IaaSystem.DailyCap;
    Check("D2_thirteenth_rejected", thirteenth, "capsules=" + state.AdCaps.Count);

    double later = 100.0 + IaaSystem.WindowSeconds + 1.0; // two oldest capsules slid out
    bool restored = iaa.OfferAd(IaaSystem.AdPlantSpeedup, later)
        && iaa.CountRecent(later) == 11; // pruned 2 (t=100,101), accepted 1 new
    Check("D3_window_slide_restores", restored,
        "recent=" + iaa.CountRecent(later) + " stored=" + state.AdCaps.Count);

    double muchLater = 100.0 + IaaSystem.WindowSeconds * 2.0 + 5.0;
    bool pruned = iaa.CapHasRoom(muchLater) && state.AdCaps.Count == 0;
    Check("D4_prune_clears_stale", pruned, "stored=" + state.AdCaps.Count);
}

// E. determinism double-run: the same cross-system script over two fresh saves
// serializes the whole profile byte-identically (codex + adcaps included).
static void GroupE()
{
    string a = PlayScript();
    string b = PlayScript();
    bool identical = string.Equals(a, b, StringComparison.Ordinal) && a.Length > 0;
    Check("E1_double_run_byte_identical", identical,
        "len=" + a.Length + " same=" + (a == b));
}

static string PlayScript()
{
    var state = new SaveState();
    var sys = new SaveSystem(state, new System.Random(Seed));
    // facade-owned codex (T010-b): single subscription, no second engine
    sys.Garden.Tick(0.0);
    string[] roster = { SpeciesTable.Bluebell, SpeciesTable.Sunburst, SpeciesTable.Roseheart, SpeciesTable.Lavendermist };
}

```

```

    for (int i = 0; i < roster.Length; i++)
    {
        sys.Garden.State.SelectedSpeciesId = roster[i];
        sys.Garden.Plant(i);
    }
    sys.Garden.Tick(6.0); // longest grow window (roseheart 6.0s) -> all four bloom
    sys.Garden.Harvest(0);
    sys.Orders.EnsureBoard();
    sys.Iaa.OfferAd(IaaSystem.AdPlantSpeedup, 100.0);
    sys.Iaa.OfferAd(IaaSystem.AdHarvestDouble, 101.0);
    sys.Iaa.OfferAd(IaaSystem.AdOrderRefresh, 102.0);
    sys.Orders.RefreshBoard();
    return JsonUtility.ToJson(state);
}

// F. save validator healing: a hand-corrupted save (null codex/adcaps lists)
//    is healed at construction and both systems keep working afterwards.
static void GroupF()
{
    var state = new SaveState();
    state.Codex = null;
    state.AdCaps = null;
    bool healed;
    try
    {
        var codex = new CodexSystem(new GardenSystem(state.Garden), state);
        var iaa = new IaaSystem(state);
        bool codexOk = state.Codex != null && codex.NoteBloom(SpeciesTable.Bluebell, 1.0)
            && codex.UnlockedCount == 1;
        bool iaaOk = state.AdCaps != null && iaa.OfferAd(IaaSystem.AdPlantSpeedup, 2.0)
            && state.AdCaps.Count == 1;
        healed = codexOk && iaaOk;
    }
    catch (Exception) { healed = false; }
    Check("F1_null_lists_healed", healed,
        "codex=" + (state.Codex != null) + " adcaps=" + (state.AdCaps != null));
}

// G. U129 policy scan + positive control: the shipped copy face and the two new
//    source files stay clean of banned policy words while the scanner is proven
//    live by a positive control hit.
static void GroupG()
{
    var scanner = new Regex(PolicyWordPattern);
    bool controlHit = scanner.IsMatch("\u653F\u7B56");

    int offenders = 0;
    int keys = 0;
    var all = LocalizeService.All();
    if (all != null)

```

```

    {
        foreach (var kv in all)
        {
            keys++;
            if (scanner.IsMatch(kv.Value)) offenders++;
        }
    }

    string runtimeDir = Path.Combine(Application.dataPath, "_Game/Scripts/Runtime");
    string[] faces = Directory.GetFiles(runtimeDir, "*.cs", SearchOption.AllDirectories);
    int sourceHits = 0;
    foreach (string file in faces)
    {
        string text = File.ReadAllText(file);
        if (scanner.IsMatch(text)) sourceHits++;
    }

    Check("G1_u129_policy_scan", controlHit && LocalizeService.Loaded && offenders == 0 && sourceHits == 0,
        "controlHit=" + controlHit + " loaded=" + LocalizeService.Loaded
        + " keys=" + keys + " offenders=" + offenders + " sources=" + faces.Length + " sourceHits=" + sourceHits);
}

// H. view mechanism law (T010-b): the three boost mechanisms land through the
// CodexView routing seam exactly as the buttons will call them.
static void GroupH()
{
    // H1: speedup halves the remaining grow window - E4 probe anchor: a fresh
    // bluebell reaches bloom at 1.50s instead of 3.0s; a half-grown one
    // keeps exactly half of what is left.
    var state = new SaveState();
    var garden = new GardenSystem(state.Garden);
    state.Garden.SelectedSpeciesId = SpeciesTable.Bluebell;
    garden.Tick(0.0);
    garden.Plant(0);
    bool freshHelped = garden.SpeedupGrowing() == 1;
    bool freshHalved = Math.Abs(garden.SecondsUntilBloom(0) - 1.5) < 1e-9;
    garden.Tick(1.5);
    bool bloomedAtHalf = state.Garden.Slots[0].Stage == SlotStage.Bloom;

    var state2 = new SaveState();
    var garden2 = new GardenSystem(state2.Garden);
    state2.Garden.SelectedSpeciesId = SpeciesTable.Bluebell;
    garden2.Tick(0.0);
    garden2.Plant(1);
    garden2.Tick(1.0); // 2.0s left
    garden2.SpeedupGrowing();
    bool midHalved = Math.Abs(garden2.SecondsUntilBloom(1) - 1.0) < 1e-9;

    var empty = new GardenSystem(new SaveState().Garden);
    bool emptyRefused = empty.SpeedupGrowing() == 0;

```

```

Check("H1_speedup_halves_remaining", freshHelped && freshHalved && bloomedAtHalf && midHalved && emptyRefused,
    "freshRemain=1.5s bloomAt=T+1.5 midRemain=1.0s emptyRefused=" + emptyRefused);

// H2: double harvest is one-shot, doubles the payout base before the
//      wallet clip, and never survives the harvest it boosted.
var state3 = new SaveState();
var garden3 = new GardenSystem(state3.Garden);
state3.Garden.SelectedSpeciesId = SpeciesTable.Bluebell;
garden3.Tick(0.0);
garden3.Plant(2);
garden3.Tick(3.0); // bluebell blooms (nominal 1 petal)
bool doubleSet = CodexView.ApplyBoostEffect(IaaSystem.AdHarvestDouble, garden3, null)
    && garden3.HarvestDoublePending;
var first = garden3.Harvest(2);
bool firstDoubled = first.Ok && first.Gained == 2 && !garden3.HarvestDoublePending;
garden3.Plant(3);
garden3.Tick(6.0); // second bloom, boost already spent
var second = garden3.Harvest(3);
bool secondNormal = second.Ok && second.Gained == 1;

var state4 = new SaveState();
var garden4 = new GardenSystem(state4.Garden);
state4.Garden.SelectedSpeciesId = SpeciesTable.Roseheart;
state4.Garden.Petals = 998; // wallet cap must still clip a doubled payout
garden4.Tick(0.0);
garden4.Plant(4);
garden4.Tick(6.0); // roseheart blooms (nominal 3)
CodexView.ApplyBoostEffect(IaaSystem.AdHarvestDouble, garden4, null);
var clipped = garden4.Harvest(4);
bool capClip = clipped.Ok && clipped.Gained == 1
    && state4.Garden.Petals == GardenSystem.WalletCap;
Check("H2_double_harvest_one_shot", doubleSet && firstDoubled && secondNormal && capClip,
    "first=" + first.Gained + " second=" + second.Gained + " capClip=" + clipped.Gained +
    " wallet=" + state4.Garden.Petals);

// H3: the refresh placement reroutes the board to a full legal board and
//      unknown placements route nowhere.
var state5 = new SaveState();
var sys5 = new SaveSystem(state5, new System.Random(Seed));
sys5.Orders.EnsureBoard();
bool refreshed = CodexView.ApplyBoostEffect(IaaSystem.AdOrderRefresh, sys5.Garden, sys5.Orders);
bool boardLegal = sys5.Orders.State.Board.Count == OrderSystem.BoardSize;
for (int i = 0; i < sys5.Orders.State.Board.Count && boardLegal; i++)
    boardLegal = OrderTable.Get(sys5.Orders.State.Board[i].DefId) != null;
bool unknownRefused = !CodexView.ApplyBoostEffect("AD_INTERSTITIAL", sys5.Garden, sys5.Orders)
    && !CodexView.ApplyBoostEffect(IaaSystem.AdOrderRefresh, sys5.Garden, null);
Check("H3_refresh_reroutes_board", refreshed && boardLegal && unknownRefused,
    "refreshed=" + refreshed + " board=" + sys5.Orders.State.Board.Count + " unknown/null refused=" + unknownRefused);

// H4: the save facade wires codex + iaa live - a bloom through the facade

```

```

        //      path auto-unlocks the codex entry (offline + live ride the same Tick).
        var state6 = new SaveState();
        var sys6 = new SaveSystem(state6, new System.Random(Seed));
        bool wired = sys6.Codex != null && sys6.Iaa != null;
        sys6.Garden.State.SelectedSpeciesId = SpeciesTable.Bluebell;
        sys6.Garden.Tick(0.0);
        sys6.Garden.Plant(5);
        sys6.Garden.Tick(3.0);
        bool autoUnlock = wired && sys6.State.Codex.Count == 1
            && string.Equals(sys6.State.Codex[0].SpeciesId, SpeciesTable.Bluebell, StringComparison.Ordinal)
            && sys6.State.Codex[0].BloomCount == 1;
        bool ledgerLive = wired && sys6.Iaa.OfferAd(IaaSystem.AdPlantSpeedup, 50.0)
            && sys6.State.AdCaps.Count == 1;
        Check("H4_facade_wires_codex_iaa", autoUnlock && ledgerLive,
            "codex=" + sys6.State.Codex.Count + " blooms=" + (sys6.State.Codex.Count > 0 ? sys6.State.Codex[0].BloomCount : 0)
            + " adcaps=" + sys6.State.AdCaps.Count);
    }
}

// ===== FILE: Assets/_Game/Scripts/Editor/G19AudioSetup.cs =====
using System.Collections.Generic;
using System.IO;
using UnityEditor;
using UnityEngine;
using Sy.BloomHaven.Audio;

namespace Sy.BloomHaven
{
    /// <summary>
    /// T008 audio catalog builder (G11 WorkerAudioSetup paradigm): scans
    /// Assets/_Game/Art/Audio for delivered clips and (re)builds
    /// Assets/_Game/Resources/G19AudioCatalog.asset so the runtime host
    /// resolves real clips by stem with null-safe fallback. With the T001 TJ
    /// batch still pending the catalog is born EMPTY (zero entries is the
    /// honest pending face - the gate asserts wiring, not asset presence).
    /// Machine face: Logs/g19_audio_catalog_report.txt.
    /// Batchmode entry: -executeMethod Sy.BloomHaven.G19AudioSetup.BuildAndSave
    /// </summary>
    public static class G19AudioSetup
    {
        const string AudioRoot = "Assets/_Game/Art/Audio";
        const string ResourcesDir = "Assets/_Game/Resources";
        const string CatalogPath = "Assets/_Game/Resources/G19AudioCatalog.asset";
        const string ReportPath = "Logs/g19_audio_catalog_report.txt";

        /// <summary>Scan Art/Audio and (re)build the catalog asset; returns
        /// the number of clips registered (0 = honest pending face).</summary>
        public static int BuildCatalog()
        {
            var entries = new List<G19AudioCatalog.Entry>();

```

(middle pages omitted)

pages 31 - 182 of 212 omitted per first-30 + last-30 excerpt rule.


```

{
    if (_orders == null)
    {
        TryInit();
        if (_orders == null) return; // garden page Start has not built its system yet
    }
    // T010-b: the codex boost can refresh the board from outside this
    // page - a stamp drift (count/ids/accepted) resyncs the card faces.
    string stamp = BoardStamp();
    if (!string.Equals(stamp, _lastBoardStamp, StringComparison.Ordinal))
    {
        _lastBoardStamp = stamp;
        SyncCards();
    }
    PollSatisfied();
    DrivePayouts(Time.unscaledDeltaTime);
}

string BoardStamp()
{
    var sb = new System.Text.StringBuilder();
    var board = _orders.State.Board;
    sb.Append(board.Count).Append(':');
    for (int i = 0; i < board.Count; i++)
        sb.Append(board[i].DefId).Append(board[i].Accepted ? '+' : '-');
    return sb.ToString();
}

// The board judges the SAME garden the S01 page owns; if the garden
// view was created this frame its Start has not run yet - retry next
// frame instead of caching a null system.
void TryInit()
{
    _garden = GardenView.EnsureInstalled();
    if (_garden.System == null) return;
    _orders = SaveHost.Current.Orders; // T007 slice b: facade engine over save.orders (no second engine, no copy-drift)
    _orders.EnsureBoard(); // page-open top-up (M4: no timer ever mutates the board)
    BuildUi();
    SyncCards();
}

// ----- live satisfaction polling (judgment is state truth) -----

void PollSatisfied()
{
    bool changed = false;
    for (int i = 0; i < _cards.Count; i++)
    {
        if (string.IsNullOrEmpty(_cards[i].DefId)) continue;
        bool satisfied = _orders.IsSatisfied(_cards[i].DefId);
    }
}

```

```

        if (satisfied == _lastSatisfied[i]) continue;
        if (satisfied) SatisfiedTransitions++;
        _lastSatisfied[i] = satisfied;
        changed = true;
    }
    if (changed) RefreshCards();
}

// ----- interactions -----

void OnAcceptTapped(CardFace card)
{
    AcceptTaps++;
    _orders.AcceptOrder(card.DefId); // commitment face; judgment stays state truth
    RefreshCards();
}

void OnDropTapped(CardFace card)
{
    DropTaps++;
    _orders.AbandonOrder(card.DefId); // R112: zero penalty, zero fee
    _orders.EnsureBoard();
    SyncCards();
}

void OnCompleteTapped(CardFace card)
{
    CompleteTaps++;
    OrderResult result = _orders.CompleteOrder(card.DefId);
    if (!result.Ok)
    {
        RefreshCards();
        return;
    }
    LastCompleteGained = result.Gained;
    LastCompleteFlipped = result.FlippedCount;
    AudioHost.PlayOrderChime(); // T008: order-complete chime on the real pay path
    _garden.RefreshWallet(); // petals landed in the shared wallet HUD
    OrderDef def = OrderTable.Get(card.DefId);
    if (def != null)
        _garden.PlayBlockFlipBeat(OrderTable.GetBlockSlots(def.FlipBlock));
    SpawnPayout(card.Bg.rectTransform.anchoredPosition.x, result.Gained);
    _orders.EnsureBoard();
    SyncCards();
}

// ----- card sync (board list <-> card faces) -----

void SyncCards()
{

```

```

    for (int i = 0; i < _cards.Count; i++)
    {
        CardFace card = _cards[i];
        bool live = i < _orders.State.Board.Count;
        card.Bg.gameObject.SetActive(live);
        if (!live)
        {
            card.DefId = string.Empty;
            _lastSatisfied[i] = false;
            continue;
        }
        card.DefId = _orders.State.Board[i].DefId;
        _lastSatisfied[i] = _orders.IsSatisfied(card.DefId);
    }
    RefreshCards();
    _lastBoardStamp = BoardStamp(); // stamp at sync so the drift check never double-fires
}

void RefreshCards()
{
    for (int i = 0; i < _cards.Count; i++)
    {
        CardFace card = _cards[i];
        if (string.IsNullOrEmpty(card.DefId)) continue;
        OrderDef def = OrderTable.Get(card.DefId);
        if (def == null) continue;
        bool accepted = _orders.State.Board[i].Accepted;
        bool satisfied = _lastSatisfied[i];
        if (card.Name != null) card.Name.text = LocalizeService.Get(def.NameKey);
        if (card.Target != null) card.Target.text = DescribeTarget(def);
        if (card.Reward != null) card.Reward.text = LocalizeService.GetF("order.reward", def.RewardPetals);
        if (card.Status != null)
        {
            if (satisfied)
            {
                card.Status.text = LocalizeService.Get("order.ready");
                card.Status.color = ReadyColor; // the order bell face
            }
            else
            {
                card.Status.text = LocalizeService.Get(accepted ? "order.accepted" : "order.offered");
                card.Status.color = accepted ? AcceptedColor : OfferedColor;
            }
        }
        if (card.Complete != null)
        {
            card.Complete.interactable = satisfied;
            card.Complete.image.color = satisfied ? ReadyColor : DimColor;
        }
    }
}

```

```

    }

    public static string DescribeTarget(OrderDef def)
    {
        OrderTarget target = def.Target;
        switch (target.Kind)
        {
            case OrderTargetKind.ColorFamily:
                return LocalizeService.GetF("order.target.color", target.MinSlots,
                    LocalizeService.Get("color." + target.Primary));
            case OrderTargetKind.Species:
                SpeciesDef species = SpeciesTable.Get(target.Primary);
                return LocalizeService.GetF("order.target.species", target.MinSlots,
                    species == null ? target.Primary : LocalizeService.Get(species.NameKey));
            case OrderTargetKind.DualColor:
                return LocalizeService.GetF("order.target.dual",
                    LocalizeService.Get("color." + target.Primary),
                    LocalizeService.Get("color." + target.Secondary), target.MinSlots);
            default:
                return string.Empty;
        }
    }
}

// ----- payout float (harvest-family feedback) -----

void SpawnPayout(float cardX, int gained)
{
    if (_safeRoot == null || _payouts.Count >= 4) return;
    var go = new GameObject("OrderPayout", typeof(RectTransform), typeof(Text));
    go.transform.SetParent(_safeRoot, false);
    var rt = (RectTransform)go.transform;
    rt.anchoredPosition = new Vector2(cardX, CardRowY + 130f);
    var t = go.GetComponent<Text>();
    t.font = Resources.GetBuiltinResource<Font>("LegacyRuntime.ttf");
    t.fontSize = 28;
    t.alignment = TextAnchor.MiddleCenter;
    t.color = ReadyColor;
    t.raycastTarget = false;
    rt.sizeDelta = new Vector2(200f, 40f);
    t.text = LocalizeService.GetF("garden.payout", gained);
    _payouts.Add(new PayoutFace
    {
        Face = t,
        Remaining = PayoutSeconds,
        BaseX = rt.anchoredPosition.x,
        BaseY = rt.anchoredPosition.y
    });
}

void DrivePayouts(float dt)

```

```

{
    for (int i = _payouts.Count - 1; i >= 0; i--)
    {
        PayoutFace payout = _payouts[i];
        payout.Remaining -= dt;
        if (payout.Remaining <= 0f)
        {
            Destroy(payout.Face.gameObject);
            _payouts.RemoveAt(i);
            continue;
        }
        float p = 1f - payout.Remaining / PayoutSeconds;
        payout.Face.rectTransform.anchoredPosition =
            new Vector2(payout.BaseX, payout.BaseY + 70f * p);
        payout.Face.color = new Color(ReadyColor.r, ReadyColor.g, ReadyColor.b, 1f - p);
    }
}

// ----- UI construction (programmatic, zero scene authoring) -----

void BuildUi()
{
    var canvasGo = new GameObject("OrderCanvas", typeof(Canvas), typeof(CanvasScaler),
        typeof(GraphicRaycaster));
    canvasGo.transform.SetParent(transform, false); // page root owns the canvas
    var canvas = canvasGo.GetComponent<Canvas>();
    canvas.renderMode = RenderMode.ScreenSpaceOverlay;
    canvas.sortingOrder = 1; // above the garden page canvas, strip-local only
    var scaler = canvasGo.GetComponent<CanvasScaler>();
    // U184: ScaleWithScreenSize or the reference frame never drives
    // device scale (raw ConstantPixelSize = shrunk UI on phone).
    scaler.uiScaleMode = CanvasScaler.ScaleMode.ScaleWithScreenSize;
    scaler.referenceResolution = new Vector2(ReferenceWidth, ReferenceHeight);

    var safeGo = new GameObject("SafeArea", typeof(RectTransform), typeof(Sy.Core.SafeAreaFitter));
    safeGo.transform.SetParent(canvasGo.transform, false);
    var safe = (RectTransform)safeGo.transform;
    safe.anchorMin = Vector2.zero;
    safe.anchorMax = Vector2.one;
    safe.sizeDelta = Vector2.zero; // full-stretch, fitter squeezes insets
    _safeRoot = safe;

    // T011-b: the wooden order board panel sits behind the strip faces
    // (AR-03, null-safe: art missing keeps the procedural strip look).
    Sy.BloomHaven.UI.PageArt.BuildNode(safe, BoardArtName, 0f, BoardArtY,
        BoardArtWidth, BoardArtHeight, Sy.Core.ArtRuntime.Get(BoardArtKey));

    var title = MakeText(safe, "BoardTitle", 30f, new Vector2(0f, -412f), 680, 36);
    AnchorTop(title.rectTransform);
    title.text = LocalizeService.Get("order.title");
}

```

```

    var hint = MakeText(safe, "BoardHint", 18f, new Vector2(0f, -444f), 680, 28);
    AnchorTop(hint.rectTransform);
    hint.color = new Color(1f, 1f, 1f, 0.75f);
    hint.text = LocalizeService.Get("order.hint");

    for (int i = 0; i < OrderSystem.BoardSize; i++) BuildCard(safe, i);
}

void BuildCard(Transform parent, int i)
{
    float x = (i - 1) * (CardWidth + 14f); // -228 / 0 / +228
    var face = new CardFace();

    var cardGo = new GameObject("OrderCard" + i, typeof(RectTransform), typeof(Image));
    cardGo.transform.SetParent(parent, false);
    var card = (RectTransform)cardGo.transform;
    card.sizeDelta = new Vector2(CardWidth, CardHeight);
    card.anchoredPosition = new Vector2(x, CardRowY);
    var bg = cardGo.GetComponent<Image>();
    bg.color = CardColor;
    bg.raycastTarget = false; // strip-local: never steal garden taps outside cards

    face.Bg = bg;
    face.Name = MakeText(cardGo.transform, "OrderName", 19f, new Vector2(0f, 70f), 204, 28);
    face.Target = MakeText(cardGo.transform, "OrderTarget", 16f, new Vector2(0f, 34f), 204, 44);
    face.Reward = MakeText(cardGo.transform, "OrderReward", 18f, new Vector2(0f, -8f), 204, 26);
    face.Reward.color = ReadyColor;
    face.Status = MakeText(cardGo.transform, "OrderStatus", 14f, new Vector2(0f, -34f), 204, 22);

    face.Accept = MakeButton(cardGo.transform, "AcceptBtn", "Accept", -74f, -68f, 64f, 34f);
    face.Accept.image.color = new Color(0.36f, 0.52f, 0.42f, 1f);
    face.Complete = MakeButton(cardGo.transform, "CompleteBtn", "Complete", 0f, -68f, 74f, 34f);
    face.Complete.image.color = DimColor;
    face.Drop = MakeButton(cardGo.transform, "DropBtn", "Drop", 72f, -68f, 56f, 34f);
    face.Drop.image.color = new Color(0.42f, 0.36f, 0.36f, 1f);

    CardFace captured = face;
    face.Accept.onClick.AddListener(() => OnAcceptTapped(captured));
    face.Drop.onClick.AddListener(() => OnDropTapped(captured));
    face.Complete.onClick.AddListener(() => OnCompleteTapped(captured));

    _cards.Add(face);
    _lastSatisfied.Add(false);
}

static Button MakeButton(Transform parent, string name, string label,
    float x, float y, float w, float h)
{
    var go = new GameObject(name, typeof(RectTransform), typeof(Image), typeof(Button));

```

```

        go.transform.SetParent(parent, false);
        var rt = (RectTransform)go.transform;
        rt.sizeDelta = new Vector2(w, h);
        rt.anchoredPosition = new Vector2(x, y);
        var button = go.GetComponent<Button>();
        button.targetGraphic = go.GetComponent<Image>(); // U184: real target graphic
        button.transition = Selectable.Transition.ColorTint;
        var text = MakeText(go.transform, "Label", 14f, Vector2.zero, (int)w - 6, (int)h - 8);
        text.text = label;
        return button;
    }

    static void AnchorTop(RectTransform rt)
    {
        rt.anchorMin = new Vector2(0.5f, 1f);
        rt.anchorMax = new Vector2(0.5f, 1f);
    }

    static Text MakeText(Transform parent, string name, float size, Vector2 pos, int w, int h)
    {
        var go = new GameObject(name, typeof(RectTransform), typeof(Text));
        go.transform.SetParent(parent, false);
        var rt = (RectTransform)go.transform;
        rt.sizeDelta = new Vector2(w, h);
        rt.anchoredPosition = pos;
        var t = go.GetComponent<Text>();
        t.font = Resources.GetBuiltinResource<Font>("LegacyRuntime.ttf");
        t.fontSize = Mathf.RoundToInt(size);
        t.alignment = TextAnchor.MiddleCenter;
        t.color = Color.white;
        t.raycastTarget = false; // never steal a button tap
        return t;
    }
}

// ===== FILE: Assets\_Game\Scripts\Runtime\Orders\OrderModel.cs =====
using System;
using System.Collections.Generic;

namespace Sy.BloomHaven.Orders
{
    /// <summary>One order instance living on the board (save.orders face).</summary>
    [Serializable]
    public sealed class OrderState
    {
        public string DefId = string.Empty;
        public bool Accepted;
    }

    /// <summary>

```

```

/// Whole order-board persistent state (save.orders face). No deadline, no expiry
/// timestamps: the board only mutates through explicit player calls, never through
/// time (offline never refreshes 欵?requirement doc S2 row).
/// </summary>
[Serializable]
public sealed class OrdersState
{
    public List<OrderState> Board = new List<OrderState>();
    public int CompletedCount;
}

/// <summary>Completion outcome; Gained is the amount actually credited (wallet cap clips it).</summary>
public readonly struct OrderResult
{
    public readonly bool Ok;
    public readonly int Gained;
    public readonly int WalletAfter;
    public readonly int FlippedCount;

    public OrderResult(bool ok, int gained, int walletAfter, int flippedCount)
    {
        Ok = ok;
        Gained = gained;
        WalletAfter = walletAfter;
        FlippedCount = flippedCount;
    }
}

}
// ===== FILE: Assets\_Game\Scripts\Runtime\Orders\OrderSystem.cs =====
using System;
using System.Collections.Generic;
using Sy.BloomHaven.Garden;

namespace Sy.BloomHaven.Orders
{
    /// <summary>
    /// Renovation order rules engine (M4). Pure C# over the garden state: the board offers
    /// pool orders, targets are judged on live (non-empty) slots, completing a satisfied
    /// order pays petals and flips its block ground, abandoning costs nothing (R112 cozy
    /// law). No deadline and no time-based refresh 欵?the board only changes through
    /// explicit calls, so offline play leaves it untouched.
    /// </summary>
    public sealed class OrderSystem
    {
        public const int BoardSize = 3; // concurrent offers on the order board

        public GardenSystem Garden { get; }
        public OrdersState State { get; }
        readonly Random _rng;
    }
}

```



```

public OrderSystem(GardenSystem garden, OrdersState state = null, Random rng = null)
{
    Garden = garden ?? throw new ArgumentNullException("garden");
    State = state ?? new OrdersState();
    _rng = rng ?? new Random();
}

/// <summary>
/// Top the board back up to BoardSize with pool orders not already shown
/// (page-open driven, idempotent when full).
/// </summary>
public void EnsureBoard()
{
    while (State.Board.Count < BoardSize)
    {
        OrderDef def = DrawNewDef();
        if (def == null) break; // pool exhausted before the board filled (cannot happen: 12 > 3)
        State.Board.Add(new OrderState { DefId = def.Id });
    }
}

/// <summary>
/// Full re-roll of the board (AD_ORDER_REFRESH mechanism face; the rewarded-ad gate
/// itself is S2 wiring 欵the data roll is free).
/// </summary>
public void RefreshBoard()
{
    State.Board.Clear();
    EnsureBoard();
}

/// <summary>Accept an offered order (commitment face for the board UI; any number may be in flight).</summary>
public bool AcceptOrder(string defId)
{
    OrderState order = FindOnBoard(defId);
    if (order == null) return false;
    order.Accepted = true;
    return true;
}

/// <summary>Live judgment for a pool order id; unknown ids are never satisfied.</summary>
public bool IsSatisfied(string defId)
{
    OrderDef def = OrderTable.Get(defId);
    return def != null && Judge(def.Target);
}

/// <summary>
/// Complete a satisfied board order: pay petals (clip at wallet cap), flip its block
/// ground to renovated, drop it from the board. Unknown, off-board or unsatisfied

```

```

/// orders change nothing. Acceptance is not required 欽?judgment is state truth.
/// </summary>
public OrderResult CompleteOrder(string defId)
{
    OrderDef def = OrderTable.Get(defId);
    if (def == null || !IsOnBoard(defId) || !Judge(def.Target))
        return new OrderResult(false, 0, Garden.State.Petals, 0);

    int credited = Math.Min(GardenSystem.WalletCap - Garden.State.Petals, def.RewardPetals);
    if (credited < 0) credited = 0;
    Garden.State.Petals += credited;

    int flipped = 0;
    int[] blockSlots = OrderTable.GetBlockSlots(def.FlipBlock);
    for (int i = 0; i < blockSlots.Length; i++)
    {
        SlotState slot = Garden.State.Slots[blockSlots[i]];
        if (!slot.Renovated) { slot.Renovated = true; flipped++; }
    }

    RemoveFromBoard(defId);
    State.CompletedCount++;
    return new OrderResult(true, credited, Garden.State.Petals, flipped);
}

/// <summary>
/// R112 cozy law: abandoning an order costs nothing and penalizes nothing 欽?no petal
/// fee, no garden damage, no lockout. The board slot stays empty until topped up.
/// </summary>
public bool AbandonOrder(string defId)
{
    return RemoveFromBoard(defId);
}

public bool IsOnBoard(string defId)
{
    return FindOnBoard(defId) != null;
}

OrderState FindOnBoard(string defId)
{
    for (int i = 0; i < State.Board.Count; i++)
        if (string.Equals(State.Board[i].DefId, defId, StringComparison.Ordinal))
            return State.Board[i];
    return null;
}

bool RemoveFromBoard(string defId)
{
    for (int i = 0; i < State.Board.Count; i++)

```

```

    {
        if (!string.Equals(State.Board[i].DefId, defId, StringComparison.Ordinal)) continue;
        State.Board.RemoveAt(i);
        return true;
    }
    return false;
}

OrderDef DrawNewDef()
{
    var free = new List<OrderDef>();
    for (int i = 0; i < OrderTable.PoolCount; i++)
    {
        OrderDef def = OrderTable.Get(i);
        if (def == null || IsOnBoard(def.Id)) continue;
        free.Add(def);
    }
    return free.Count == 0 ? null : free[_rng.Next(free.Count)];
}

/// <summary>
/// Judge a target against live slots: alive flowers (growing or bloom) count toward
/// targets; dual-color needs at least one of each family and the combined minimum.
/// </summary>
bool Judge(OrderTarget target)
{
    int primary = 0, secondary = 0;
    SlotState[] slots = Garden.State.Slots;
    for (int i = 0; i < slots.Length; i++)
    {
        if (slots[i].Stage == SlotStage.Empty) continue;
        SpeciesDef species = SpeciesTable.Get(slots[i].SpeciesId);
        if (species == null) continue;
        if (Matches(target, species, true)) primary++;
        if (target.Kind == OrderTargetKind.DualColor && Matches(target, species, false)) secondary++;
    }
    switch (target.Kind)
    {
        case OrderTargetKind.ColorFamily:
        case OrderTargetKind.Species:
            return primary >= target.MinSlots;
        case OrderTargetKind.DualColor:
            return primary >= 1 && secondary >= 1 && primary + secondary >= target.MinSlots;
        default:
            return false;
    }
}

static bool Matches(OrderTarget target, SpeciesDef species, bool primaryFace)
{

```

```

        switch (target.Kind)
        {
            case OrderTargetKind.ColorFamily:
                return string.Equals(species.ColorFamily, target.Primary, StringComparison.Ordinal);
            case OrderTargetKind.Species:
                return string.Equals(species.Id, target.Primary, StringComparison.Ordinal);
            case OrderTargetKind.DualColor:
                string want = primaryFace ? target.Primary : target.Secondary;
                return string.Equals(species.ColorFamily, want, StringComparison.Ordinal);
            default:
                return false;
        }
    }
}

// ===== FILE: Assets\_Game\Scripts\Runtime\Orders\OrderTable.cs =====
using System;

namespace Sy.BloomHaven.Orders
{
    /// <summary>Target shape of a renovation order (requirement doc order schema: target_spec).</summary>
    public enum OrderTargetKind
    {
        ColorFamily,    // at least MinSlots alive flowers of one color family
        Species,        // at least MinSlots alive flowers of one species
        DualColor       // both color families present, combined at least MinSlots
    }

    /// <summary>Order target_spec face (frozen in OrderTable, judged by OrderSystem).</summary>
    public sealed class OrderTarget
    {
        public readonly OrderTargetKind Kind;
        public readonly string Primary;    // color family id, or species id for Species targets
        public readonly string Secondary;  // second color family id (DualColor only)
        public readonly int MinSlots;

        public OrderTarget(OrderTargetKind kind, string primary, string secondary, int minSlots)
        {
            Kind = kind;
            Primary = primary;
            Secondary = secondary;
            MinSlots = minSlots;
        }
    }

    /// <summary>
    /// Renovation order definition, frozen pool of 12 (requirement doc L79 anchor).
    /// Rewards sit in the 3-8 petals band; difficulty slopes single-target -> dual-target.
    /// </summary>
    public sealed class OrderDef

```

```

{
    public readonly string Id;
    public readonly string NameEn;
    public readonly string NameKey;
    public readonly OrderTarget Target;
    public readonly int RewardPetals;
    public readonly int FlipBlock;

    public OrderDef(string id, string nameEn, string nameKey, OrderTarget target,
        int rewardPetals, int flipBlock)
    {
        Id = id;
        NameEn = nameEn;
        NameKey = nameKey;
        Target = target;
        RewardPetals = rewardPetals;
        FlipBlock = flipBlock;
    }
}

/// <summary>
/// Frozen order pool (M4 renovation orders). The 16 preset slots form four renovation
/// blocks of four slots (2x2 quadrants of the 4x4 grid); completing an order flips the
/// ground of its block from wasteland to flower bed. No deadline by design (cozy law).
/// </summary>
public static class OrderTable
{
    public const string Ord01 = "ORD-01";
    public const string Ord12 = "ORD-12";
    public const int PoolCount = 12;
    public const int BlockCount = 4;        // 16 slots = 4 renovation blocks
    public const int SlotsPerBlock = 4;

    static readonly OrderDef[] Pool =
    {
        new OrderDef(Ord01, "Bluebell Bed", "order_ORD01",
            new OrderTarget(OrderTargetKind.ColorFamily, "blue", null, 3), 3, 0),
        new OrderDef("ORD-02", "Sunshelf", "order_ORD02",
            new OrderTarget(OrderTargetKind.Species, "SP02", null, 2), 3, 0),
        new OrderDef("ORD-03", "Blue & Gold Blend", "order_ORD03",
            new OrderTarget(OrderTargetKind.DualColor, "blue", "yellow", 5), 5, 0),
        new OrderDef("ORD-04", "Sunny Plot", "order_ORD04",
            new OrderTarget(OrderTargetKind.ColorFamily, "yellow", null, 3), 4, 1),
        new OrderDef("ORD-05", "Heartrow", "order_ORD05",
            new OrderTarget(OrderTargetKind.ColorFamily, "red", null, 2), 4, 1),
        new OrderDef("ORD-06", "Dusk Garden", "order_ORD06",
            new OrderTarget(OrderTargetKind.DualColor, "blue", "purple", 5), 5, 1),
        new OrderDef("ORD-07", "Mist Bed", "order_ORD07",
            new OrderTarget(OrderTargetKind.ColorFamily, "purple", null, 3), 4, 2),
        new OrderDef("ORD-08", "Rose Arbor", "order_ORD08",

```

```

        new OrderTarget(OrderTargetKind.Species, "SP03", null, 2), 5, 2),
    new OrderDef("ORD-09", "Ember Duet", "order_ORD09",
        new OrderTarget(OrderTargetKind.DualColor, "red", "purple", 5), 6, 2),
    new OrderDef("ORD-10", "Sunset Border", "order_ORD10",
        new OrderTarget(OrderTargetKind.DualColor, "yellow", "red", 6), 6, 3),
    new OrderDef("ORD-11", "Lavender Lane", "order_ORD11",
        new OrderTarget(OrderTargetKind.Species, "SP04", null, 3), 6, 3),
    new OrderDef(Ord12, "Starfield Mosaic", "order_ORD12",
        new OrderTarget(OrderTargetKind.DualColor, "blue", "red", 8), 8, 3),
};

/// <summary>Index lookup (0..PoolCount-1); out of range returns null.</summary>
public static OrderDef Get(int index) =>
    (index >= 0 && index < Pool.Length) ? Pool[index] : null;

/// <summary>Id lookup; unknown id returns null (callers treat null as invalid).</summary>
public static OrderDef Get(string id)
{
    for (int i = 0; i < Pool.Length; i++)
        if (string.Equals(Pool[i].Id, id, StringComparison.Ordinal)) return Pool[i];
    return null;
}

/// <summary>
/// Preset slots composing a renovation block (4x4 row-major grid, 2x2 quadrants).
/// Out-of-range block returns null.
/// </summary>
public static int[] GetBlockSlots(int blockIndex)
{
    switch (blockIndex)
    {
        case 0: return new[] { 0, 1, 4, 5 };
        case 1: return new[] { 2, 3, 6, 7 };
        case 2: return new[] { 8, 9, 12, 13 };
        case 3: return new[] { 10, 11, 14, 15 };
        default: return null;
    }
}
}

// ===== FILE: Assets\_Game\Scripts\Runtime\Save\SaveHost.cs =====
using System;
using UnityEngine;

namespace Sy.BloomHaven.Save
{
    /// <summary>
    /// App-lifetime save host (T007 slice b): loads + settles the save exactly once
    /// per process before any page reads state (boot wires it, pages consume
    /// Current). LastSettlement feeds the S01 welcome-back bubble. Pages drive

```

```

/// persistence through SaveNow on a periodic interval plus pause/quit hooks.
/// </summary>
public static class SaveHost
{
    static SaveSystem _current;
    public static OfflineSettlement? LastSettlement { get; private set; }

    public static SaveSystem Current => Ensure();

    /// <summary>
    /// Load-or-fresh, bridge the clock domain, settle the away window once.
    /// Fresh saves settle to zero (no free petals on first launch).
    /// </summary>
    public static SaveSystem Ensure()
    {
        if (_current != null) return _current;
        double uptimeNow = Time.realtimeSinceStartupAsDouble;
        SaveFile file = SaveStore.Load();
        if (file == null)
        {
            _current = new SaveSystem(); // fresh face: absent or unreadable save
            LastSettlement = _current.Settle(uptimeNow); // stamps the anchor, accrues nothing
            return _current;
        }
        double wallAway = Math.Max(0, SaveStore.WallSeconds() - file.WallSavedAt); // backward wall clock = 0 away, no double-pay
        _current = new SaveSystem(file.State);
        LastSettlement = SaveStore.Resume(_current, uptimeNow, file.UptimeSavedAt, wallAway);
        return _current;
    }

    /// <summary>Periodic / pause / quit persistence (live path).</summary>
    public static void SaveNow(double uptimeNow)
    {
        if (_current == null) return;
        SaveStore.Persist(_current, uptimeNow);
    }
}

}
// ===== FILE: Assets\_Game\Scripts\Runtime\Save\SaveModel.cs =====
using System;
using System.Collections.Generic;

namespace Sy.BloomHaven.Save
{
    /// <summary>Codex collection entry (save.codex face, S5). Unlocked once at first
    /// bloom; BloomCount keeps rising for the codex card (T010-a face).</summary>
    [Serializable]
    public sealed class CodexEntry
    {
        public string SpeciesId = string.Empty;
    }
}

```

```

        public double UnlockedAt;
        public int BloomCount;
    }

    /// <summary>One rewarded-ad capsule record (save.adcaps face, S6). U149: 12/day hard cap enforced by the future ad system.
</summary>
    [Serializable]
    public sealed class AdCapsule
    {
        public string Type = string.Empty;
        public double OfferedAt;
    }

    /// <summary>
    /// Whole-profile save state (M6): five faces = slots + wallet (hosted by GardenState,
    /// see its class doc) / orders / codex / adcaps, plus LastSeenAt as the monotonic-clock
    /// anchor for offline accrual (M6: away seconds -> petals, 60s -> 6, cap 10, E4 h5).
    /// </summary>
    [Serializable]
    public sealed class SaveState
    {
        public double LastSeenAt;
        public Garden.GardenState Garden = new Garden.GardenState(); // save.slots + save.wallet faces
        public Orders.OrdersState Orders = new Orders.OrdersState(); // save.orders face
        public List<CodexEntry> Codex = new List<CodexEntry>(); // save.codex face
        public List<AdCapsule> AdCaps = new List<AdCapsule>(); // save.adcaps face
    }

    /// <summary>Offline settlement outcome (M6). Raw is the uncapped accrual, Accrued is what was actually credited.</summary>
    public readonly struct OfflineSettlement
    {
        public readonly double AwaySeconds;
        public readonly int Raw;
        public readonly int Accrued;
        public readonly bool Capped;
        public readonly int WalletAfter;

        public OfflineSettlement(double awaySeconds, int raw, int accrued, bool capped, int walletAfter)
        {
            AwaySeconds = awaySeconds;
            Raw = raw;
            Accrued = accrued;
            Capped = capped;
            WalletAfter = walletAfter;
        }
    }
}

// ===== FILE: Assets\_Game\Scripts\Runtime\Save\SaveStore.cs =====
using System;
using UnityEngine;

```



```

namespace Sy.BloomHaven.Save
{
    /// <summary>
    /// On-disk save wrapper (T007 slice b). The five faces ride SaveState; the two
    /// anchors translate the uptime clock domain across process restarts:
    /// WallSavedAt is wall-clock Unix seconds at save time, UptimeSavedAt is
    /// Time.realtimeSinceStartupAsDouble at save time. Version gates decode:
    /// anything unexpected heals to a fresh save (malformed-input cure law).
    /// </summary>
    [Serializable]
    public sealed class SaveFile
    {
        public int Version = 1;
        public double WallSavedAt;
        public double UptimeSavedAt;
        public SaveState State;
    }

    /// <summary>
    /// PlayerPrefs persistence + cross-launch clock bridge (T007 slice b). The away
    /// window is measured on the wall clock (the only clock that survives a
    /// restart); growth replays through the SAME Settle path as live play by
    /// remapping PlantedAt so the maturity a slot had at save time is preserved,
    /// then letting Settle(now) advance it by exactly the away seconds. The engine
    /// clock stays in the live uptime domain, so plants made after a revisit grow
    /// on live ticks with zero future-dated drift.
    /// </summary>
    public static class SaveStore
    {
        public const string PlayerPrefsKey = "g19_save_v1";
        public const int SaveVersion = 1;

        /// <summary>Wall-clock Unix seconds (restart-surviving clock face).</summary>
        public static double WallSeconds()
            => DateTimeOffset.UtcNow.ToUnixTimeSeconds();

        /// <summary>Encode the live save into the on-disk wrapper (pure, injected clocks).</summary>
        public static string Encode(SaveSystem save, double uptimeNow, double wallNowSeconds)
        {
            save.MarkSeen(uptimeNow); // anchor stamps at the save instant
            var file = new SaveFile
            {
                Version = SaveVersion,
                WallSavedAt = wallNowSeconds,
                UptimeSavedAt = uptimeNow,
                State = save.State
            };
            return JsonUtility.ToJson(file);
        }
    }
}

```

```

/// <summary>Decode a save payload; null heals any malformed or unknown-version input.</summary>
public static SaveFile Decode(string json)
{
    if (string.IsNullOrEmpty(json)) return null;
    try
    {
        var file = JsonUtility.FromJson<SaveFile>(json);
        if (file == null || file.State == null || file.Version != SaveVersion) return null;
        if (file.WallSavedAt <= 0 || file.UptimeSavedAt < 0) return null;
        if (file.State.Garden == null || file.State.Garden.Slots == null
            || file.State.Garden.Slots.Length != Garden.GardenSystem.SlotCount) return null;
        if (file.State.Orders == null || file.State.Codex == null || file.State.AdCaps == null) return null;
        return file;
    }
    catch
    {
        return null; // malformed-input cure law: unreadable save = fresh start
    }
}

/// <summary>
/// Bridge a decoded save into the live session and settle the away window
/// through the SAME Settle path as live play (single-path law). Slots remap
/// so their maturity at save time is preserved, then Settle advances growth
/// by exactly wallAway seconds. The anchor is remapped to now-wallAway so
/// Settle's internal away equals the wall gap; a fresh face (LastSeenAt=0)
/// keeps the zero guard - no free petals on first launch.
/// </summary>
public static OfflineSettlement Resume(SaveSystem save, double uptimeNow,
    double savedUptimeAt, double wallAway)
{
    double shift = uptimeNow - savedUptimeAt - wallAway;
    var slots = save.State.Garden.Slots;
    for (int i = 0; i < slots.Length; i++) slots[i].PlantedAt += shift;
    if (save.State.LastSeenAt > 0) save.State.LastSeenAt = uptimeNow - wallAway;
    return save.Settle(uptimeNow); // accrual + offline growth + anchor stamp, one path
}

/// <summary>Persist to PlayerPrefs (live path).</summary>
public static void Persist(SaveSystem save, double uptimeNow)
{
    PlayerPrefs.SetString(PlayerPrefsKey, Encode(save, uptimeNow, WallSeconds()));
    PlayerPrefs.Save();
}

/// <summary>Load from PlayerPrefs; null when absent or malformed.</summary>
public static SaveFile Load()
    => Decode(PlayerPrefs.GetString(PlayerPrefsKey, string.Empty));
}
}

```

```
// ===== FILE: Assets\_Game\Scripts\Runtime\Save\SaveSystem.cs =====
using System;

namespace Sy.BloomHaven.Save
{
    /// <summary>
    /// Save facade + offline settlement engine (M6/T007). Owns the five-face SaveState and
    /// wires the rules engines onto it (GardenSystem over save.slots+wallet, OrderSystem over
    /// save.orders; T010-b adds CodexSystem over save.codex + IaaSystem over save.adcaps so the
    /// live app records codex unlocks on every bloom and shares one ad ledger with the boost
    /// buttons). Offline accrual: floor(away/10) petals, capped at 10 (E4 h5: 60s -> 6),
    /// then the wallet cap 999 clips the credit. Growth resume rides the same monotonic
    /// Tick path as live play (T003 law: stage math is always (now - plantedAt), no new path).
    /// Orders never mutate through time (S2 row: offline never refreshes).
    /// </summary>
    public sealed class SaveSystem
    {
        public const double AccrualSecondsPerPetal = 10.0; // 60s away -> 6 petals (E4 h5)
        public const int AccrualCap = 10; // accrual hard cap (E4 h5)

        public SaveState State { get; }
        public Garden.GardenSystem Garden { get; }
        public Orders.OrderSystem Orders { get; }
        public Codex.CodexSystem Codex { get; } // T010-b: bloom-subscribed collection engine
        public Iaa.IaaSystem Iaa { get; } // T010-b: single ad-capsule ledger face

        public SaveSystem(SaveState state = null, Random rng = null)
        {
            State = state ?? new SaveState();
            Garden = new Garden.GardenSystem(State.Garden);
            Orders = new Orders.OrderSystem(Garden, State.Orders, rng);
            Codex = new Codex.CodexSystem(Garden, State);
            Iaa = new Iaa.IaaSystem(State);
        }

        /// <summary>
        /// Settle a return after being away: accrue petals for the gap since LastSeenAt, resume
        /// offline growth via the shared Tick path, then stamp the new anchor. A fresh save
        /// (LastSeenAt == 0) settles to zero 欵no free petals on first launch.
        /// </summary>
        public OfflineSettlement Settle(double nowSeconds)
        {
            double away = State.LastSeenAt > 0 ? Math.Max(0, nowSeconds - State.LastSeenAt) : 0;
            int raw = (int)Math.Floor(away / AccrualSecondsPerPetal);
            bool capped = raw > AccrualCap;
            int accrued = Math.Min(raw, AccrualCap);
            int credited = Math.Min(Sy.BloomHaven.Garden.GardenSystem.WalletCap - State.Garden.Petals, accrued);
            if (credited < 0) credited = 0;
            State.Garden.Petals += credited;
            Garden.Tick(nowSeconds); // offline growth resume, same monotonic path as live play
        }
    }
}
```

```

        if (nowSeconds > State.LastSeenAt) State.LastSeenAt = nowSeconds; // anchor never regresses (backward clock = away 0, no
double-pay)
        return new OfflineSettlement(away, raw, credited, capped, State.Garden.Petals);
    }

    /// <summary>Stamp the anchor without accruing (periodic save / graceful quit).</summary>
    public void MarkSeen(double nowSeconds)
    {
        if (nowSeconds >= State.LastSeenAt) State.LastSeenAt = nowSeconds;
    }
}
}
// ===== FILE: Assets\_Game\Scripts\Runtime\Share\ShareCodec.cs =====
using System;
using Sy.BloomHaven.Garden;
using Sy.BloomHaven.Orders;

namespace Sy.BloomHaven.Share
{
    /// <summary>
    /// M5 offline garden share code (X032 doctrine: pure local, zero server).
    /// compose: full garden face -> "BH1|<16 slot chars>|<4 block chars>|<petals>".
    /// Slot char: '0'=empty, '1'..'4'=SP01..SP04 growing, 'A'..'D'=SP01..SP04 bloom.
    /// Block char: '0'=wasteland, '1'=renovated (uniform per block, C4 idempotence law).
    /// Petals: decimal 0..999 (wallet cap), the bragging number on the visitor side.
    /// PlantedAt is deliberately NOT encoded (time-relative); growing slots decode with
    /// PlantedAt=0 so they bloom on the visitor's own clock (living-garden entrance).
    /// SelectedSpeciesId is player-local UI state, not garden content - not encoded.
    /// DecodeShare returns null on any malformed input (cozy law: never throw at a visitor).
    /// </summary>
    public static class ShareCodec
    {
        public const string Prefix = "BH1";
        public const int MaxShareChars = 96; // north star #4 cap (requirement doc, assert p2)
        const int Sections = 4; // prefix | slots | blocks | petals

        /// <summary>Information metric for the share_compose telemetry event:
        /// log2 of the distinct encodable state space (16 slots x 9 states + 4 block bits + petals 0..999).</summary>
        public static double StateSpaceBits =>
            GardenSystem.SlotCount * Math.Log(9.0, 2.0) + OrderTable.BlockCount + Math.Log(1000.0, 2.0);

        /// <summary>Compose the shareable garden face. Never throws; unknown species encode as empty.</summary>
        public static string ComposeShare(GardenState state)
        {
            if (state == null) return null;
            var slots = new char[GardenSystem.SlotCount];
            for (int i = 0; i < slots.Length; i++)
            {
                SlotState slot = state.Slots[i];
                int speciesIndex = SpeciesIndexOf(slot == null ? string.Empty : slot.SpeciesId);
                if (speciesIndex < 0) { slots[i] = '0'; continue; }
            }
        }
    }
}

```

```

        if (slot.Stage == SlotStage.Bloom) slots[i] = (char)('A' + speciesIndex);
        else if (slot.Stage == SlotStage.Growing) slots[i] = (char)('1' + speciesIndex);
        else slots[i] = '0';
    }
    var blocks = new char[OrderTable.BlockCount];
    for (int b = 0; b < blocks.Length; b++)
    {
        int[] blockSlots = OrderTable.GetBlockSlots(b);
        SlotState head = blockSlots == null ? null : state.Slots[blockSlots[0]];
        blocks[b] = head != null && head.Renovated ? '1' : '0';
    }
    int petals = Math.Max(0, Math.Min(GardenSystem.WalletCap, state.Petals));
    return Prefix + "|" + new string(slots) + "|" + new string(blocks) + "|" + petals;
}

/// <summary>Decode a visitor-typed share code into a playable GardenState; null on malformed input.</summary>
public static GardenState DecodeShare(string code)
{
    if (string.IsNullOrEmpty(code)) return null;
    string trimmed = code.Trim();
    if (trimmed.Length == 0 || trimmed.Length > MaxShareChars) return null;
    string[] parts = trimmed.Split('|');
    if (parts.Length != Sections) return null;
    if (!string.Equals(parts[0], Prefix, StringComparison.Ordinal)) return null;
    string slotSection = parts[1];
    string blockSection = parts[2];
    string petalSection = parts[3];
    if (slotSection.Length != GardenSystem.SlotCount) return null;
    if (blockSection.Length != OrderTable.BlockCount) return null;

    var state = new GardenState();
    for (int i = 0; i < slotSection.Length; i++)
    {
        char c = slotSection[i];
        if (c == '0') continue; // empty is the constructed default
        int speciesIndex;
        SlotStage stage;
        if (c >= '1' && c <= '4') { speciesIndex = c - '1'; stage = SlotStage.Growing; }
        else if (c >= 'A' && c <= 'D') { speciesIndex = c - 'A'; stage = SlotStage.Bloom; }
        else return null;
        SpeciesDef def = SpeciesTable.Get(speciesIndex);
        if (def == null) return null;
        state.Slots[i].SpeciesId = def.Id;
        state.Slots[i].Stage = stage;
        state.Slots[i].PlantedAt = 0; // visitor clock owns growth from now (living-garden entrance)
    }
    for (int b = 0; b < blockSection.Length; b++)
    {
        char c = blockSection[b];
        if (c != '0' && c != '1') return null;
    }
}

```

```

        if (c != '1') continue;
        int[] blockSlots = OrderTable.GetBlockSlots(b);
        if (blockSlots == null) return null;
        for (int s = 0; s < blockSlots.Length; s++)
            state.Slots[blockSlots[s]].Renovated = true;
    }
    if (petalSection.Length < 1 || petalSection.Length > 3) return null;
    for (int i = 0; i < petalSection.Length; i++)
        if (petalSection[i] < '0' || petalSection[i] > '9') return null;
    int petals;
    if (!int.TryParse(petalSection, out petals)) return null;
    if (petals < 0 || petals > GardenSystem.WalletCap) return null;
    state.Petals = petals;
    return state;
}

/// <summary>Species roster index 0..3; -1 when the id is unknown or empty.</summary>
static int SpeciesIndexOf(string speciesId)
{
    for (int i = 0; i < SpeciesTable.Count; i++)
    {
        SpeciesDef def = SpeciesTable.Get(i);
        if (def != null && string.Equals(def.Id, speciesId, StringComparison.Ordinal)) return i;
    }
    return -1;
}
}

}

// ===== FILE: Assets\Game\Scripts\Runtime\Share\ShareView.cs =====
// G19 share page host (T006): S04_Share overlay - the ShareCodec consumer.
// Opens from the S01 share entry button. Shows the garden's live share code,
// copies it to the system clipboard, and decodes a friend's code into the
// visitor garden (living-garden entrance: growing slots keep PlantedAt=0
// so they bloom on the visitor clock). Malformed input never throws (cozy
// law) - the status face says the code was not recognized and play goes on.
// Copy stays English-only on LegacyRuntime for now; the CJK font chain swap
// is a separate batch per doc-11 (U184 risk noted).
using System;
using Sy.BloomHaven.Boot;
using Sy.BloomHaven.Garden;
using Sy.BloomHaven.Localize;
using UnityEngine;
using UnityEngine.EventSystems;
using UnityEngine.SceneManagement;
using UnityEngine.UI;

namespace Sy.BloomHaven.Share
{
    public sealed class ShareView : MonoBehaviour
    {

```

```

public const string PageRootName = "S04_Share"; // T006 page-list name
public const int PageSortOrder = 2;           // above garden(0)/board(1), below guide(3)
public const float ReferenceWidth = 720f;
public const float ReferenceHeight = 1280f;

// element contract (PagesCheck zero-missing law, requirement doc S04 face)
public const string CodeFaceName = "ShareCodeFace";
public const string CopyButtonName = "CopyButton";
public const string DecodeFieldName = "DecodeField";
public const string VisitButtonName = "VisitButton";
public const string StatusFaceName = "ShareStatus";
public const string CloseButtonName = "CloseButton";

// S4 AIGC transparency note (T012-c, doc-19 sec-5): AI-assisted art and
// audio declaration line at the share-page bottom.
public const string AigcNoteName = "AigcNote";
public const string AigcNoteKey = "share.aigc_note";

// T011-b reskin contract: the share card frame (AR-06) backs the
// code column; art missing keeps the modal's dim backdrop.
public const string CardArtName = "ShareCardArt";
public const string CardArtKey = "share_card_frame";
public const float CardArtWidth = 540f; // 9:16 portrait of the 750x1334 spec
public const float CardArtHeight = 960f;
public const float CardArtY = 150f; // centers the code column on the card

// status law faces (cozy copy, assertable by name at PagesCheck;
// values resolve through cfg_i18n keys - T008 U114 L0)
public const string InvalidCodeKey = "share.invalid_code";
public const string VisitedKey = "share.visited";
public const string CopiedKey = "share.copied";
public static readonly string InvalidCodeMessage = LocalizeService.Get(InvalidCodeKey);
public static readonly string VisitedMessage = LocalizeService.Get(VisitedKey);
public static readonly string CopiedMessage = LocalizeService.Get(CopiedKey);
public const float VisitCloseDelaySeconds = 1.2f; // welcome line hold, then step in;

// counters for future play-mode gates (redline11-style assertions)
public int CopyTaps;
public int VisitTaps;
public int LastDecodeOk = -1; // -1 = no attempt yet, 0 = rejected, 1 = visited
public bool IsOpen;

GardenView _garden;
Text _codeFace;
Text _statusFace;
InputField _decodeField;
GameObject _panel;
bool _built;
float _autoCloseAt = -1f; // visit success: show the welcome line, then step in

```

```

static bool _subscribed;

/// <summary>
/// Wire the share page onto S19_Main as soon as it finishes loading
/// (same static-installer family as GardenView: the boot host is
/// destroyed by the single-mode switch before its coroutine resumes).
/// The panel starts hidden - S01 owns the screen until the entry tap.
/// </summary>
public static void InstallOnSceneLoad()
{
    if (_subscribed) return;
    _subscribed = true;
    SceneManager.sceneLoaded += OnSceneLoaded;
}

static void OnSceneLoaded(Scene scene, LoadSceneMode mode)
{
    if (scene.name != SceneIds.Main) return;
    EnsureInstalled();
}

public static ShareView EnsureInstalled()
{
    ShareView view = (ShareView)FindObjectOfType(typeof(ShareView));
    if (view != null) return view;
    // input floor (U184 family lesson): programmatic UGUI taps die
    // without an EventSystem in the loaded scene.
    if (FindObjectOfType(typeof(EventSystem)) == null)
        new GameObject("EventSystem", typeof(EventSystem), typeof(StandaloneInputModule));
    var go = new GameObject(PageRootName, typeof(ShareView));
    return go.GetComponent<ShareView>();
}

/// <summary>Gate seam (PagesCheck group C): the code face always
/// renders exactly what the codec composes for the live garden.</summary>
public static string ComposeCurrentCode(GardenState state)
{
    return ShareCodec.ComposeShare(state);
}

void Start() => Initialize();

/// <summary>Headless boot seam (U085 slice-B patrol): the full Start()
/// face callable from editor gates (see GardenView.Initialize).</summary>
public void Initialize()
{
    _garden = GardenView.EnsureInstalled();
    BuildUi();
    SetOpen(false);
}

```



```

// ----- open/close -----

/// <summary>Entry toggle from the S01 share button.</summary>
public void Toggle()
{
    SetOpen(!IsOpen);
}

void SetOpen(bool open)
{
    IsOpen = open;
    _autoCloseAt = -1f; // any manual open/close cancels a pending auto-close
    if (_panel == null) return;
    _panel.SetActive(open);
    if (open)
    {
        RefreshCode();
        if (_statusFace != null) _statusFace.text = string.Empty;
        if (_decodeField != null) _decodeField.text = string.Empty;
    }
}

void RefreshCode()
{
    if (_codeFace == null || _garden == null || _garden.System == null) return;
    _codeFace.text = ComposeCurrentCode(_garden.System.State);
}

// ----- interactions -----

void OnCopyTapped()
{
    CopyTaps++;
    if (_garden == null || _garden.System == null) return;
    string code = ComposeCurrentCode(_garden.System.State);
    if (string.IsNullOrEmpty(code)) return;
    GUIUtility.systemCopyBuffer = code;
    if (_statusFace != null) _statusFace.text = CopiedMessage;
}

void OnVisitTapped()
{
    VisitTaps++;
    if (_decodeField == null) return;
    GardenState visitor = ShareCodec.DecodeShare(_decodeField.text);
    if (visitor == null)
    {
        LastDecodeOk = 0;
        if (_statusFace != null) _statusFace.text = InvalidCodeMessage;
    }
}

```

```

        return;
    }
    LastDecodeOk = 1;
    _garden.LoadVisitorGarden(visitor); // in-place copy, board stays live
    if (_statusFace != null) _statusFace.text = VisitedMessage;
    _autoCloseAt = Time.unscaledTime + VisitCloseDelaySeconds; // welcome line first, then step in
}

void Update()
{
    if (IsOpen && _autoCloseAt >= 0f && Time.unscaledTime >= _autoCloseAt)
        SetOpen(false);
}

void OnCloseTapped()
{
    SetOpen(false);
}

// ----- UI construction (programmatic, zero scene authoring) -----

void BuildUi()
{
    if (_built) return;
    _built = true;
    var canvasGo = new GameObject("ShareCanvas", typeof(Canvas), typeof(CanvasScaler),
        typeof(GraphicRaycaster));
    canvasGo.transform.SetParent(transform, false); // page root owns the canvas
    var canvas = canvasGo.GetComponent<Canvas>();
    canvas.renderMode = RenderMode.ScreenSpaceOverlay;
    canvas.sortingOrder = PageSortOrder; // overlay law: garden 0 -> board 1 -> share 2
    var scaler = canvasGo.GetComponent<CanvasScaler>();
    // U184: ScaleWithScreenSize or the reference frame never drives
    // device scale (raw ConstantPixelSize = shrunken UI on phone).
    scaler.uiScaleMode = CanvasScaler.ScaleMode.ScaleWithScreenSize;
    scaler.referenceResolution = new Vector2(ReferenceWidth, ReferenceHeight);

    _panel = new GameObject("SharePanel", typeof(RectTransform), typeof(Image));
    _panel.transform.SetParent(canvasGo.transform, false);
    var panel = (RectTransform)_panel.transform;
    panel.anchorMin = Vector2.zero;
    panel.anchorMax = Vector2.one;
    panel.sizeDelta = Vector2.zero; // full-screen dim backdrop
    panel.anchoredPosition = Vector2.zero;
    var backdrop = _panel.GetComponent<Image>();
    backdrop.color = new Color(0.08f, 0.06f, 0.10f, 0.82f);
    backdrop.raycastTarget = true; // modal: swallow taps behind the panel

    // T011-b: the share card frame (AR-06) backs the code column,
    // null-safe - art missing keeps the dim-backdrop face.

```

```

Sy.BloomHaven.UI.PageArt.BuildNode(_panel.transform, CardArtName, 0f, CardArtY,
    CardArtWidth, CardArtHeight, Sy.Core.ArtRuntime.Get(CardArtKey));

var title = MakeText(_panel.transform, "ShareTitle", 32f, new Vector2(0f, 430f), 680, 44);
title.text = LocalizeService.Get("share.title");

var codeLabel = MakeText(_panel.transform, "CodeLabel", 18f, new Vector2(0f, 372f), 680, 28);
codeLabel.color = new Color(1f, 1f, 1f, 0.75f);
codeLabel.text = LocalizeService.Get("share.code_label");

_codeFace = MakeText(_panel.transform, CodeFaceName, 24f, new Vector2(0f, 318f), 660, 120);
_codeFace.color = new Color(0.85f, 0.95f, 1f, 1f);
_codeFace.horizontalOverflow = HorizontalWrapMode.Wrap;

var copy = MakeButton(_panel.transform, CopyButtonName, "Copy code", 0f, 228f, 190f, 52f);
copy.image.color = new Color(0.36f, 0.52f, 0.42f, 1f);
copy.onClick.AddListener(OnCopyTapped);

var visitLabel = MakeText(_panel.transform, "VisitLabel", 18f, new Vector2(0f, 150f), 680, 28);
visitLabel.color = new Color(1f, 1f, 1f, 0.75f);
visitLabel.text = LocalizeService.Get("share.visit_label");

_decodeField = MakeDecodeField(_panel.transform, 88f);
_decodeField.text = string.Empty;

var visit = MakeButton(_panel.transform, VisitButtonName, "Visit", 0f, 18f, 190f, 52f);
visit.image.color = new Color(0.42f, 0.56f, 0.86f, 1f);
visit.onClick.AddListener(OnVisitTapped);

_statusFace = MakeText(_panel.transform, StatusFaceName, 18f, new Vector2(0f, -42f), 660, 30);
_statusFace.color = new Color(1f, 0.85f, 0.45f, 1f);
_statusFace.text = string.Empty;

var close = MakeButton(_panel.transform, CloseButtonName, "Close", 0f, -110f, 140f, 46f);
close.image.color = new Color(0.42f, 0.36f, 0.36f, 1f);
close.onClick.AddListener(OnCloseTapped);

// S4 AIGC transparency note (T012-c): doc-19 sec-5 declaration line.
// Art and audio assets are AI-generated; the note rides the share
// page (the game's community surface), value via cfg_i18n key.
var aigcNote = MakeText(_panel.transform, AigcNoteName, 13f, new Vector2(0f, -168f), 680, 24);
aigcNote.color = new Color(1f, 1f, 1f, 0.55f);
aigcNote.text = LocalizeService.Get(AigcNoteKey);
}

/// <summary>"Paste a friend's code" input line: background image +
/// placeholder + text component, per UGUI InputField requirements.</summary>
InputField MakeDecodeField(Transform parent, float y)
{
    var fieldGo = new GameObject(DecodeFieldName, typeof(RectTransform), typeof(Image), typeof(InputField));

```

```

    fieldGo.transform.SetParent(parent, false);
    var rt = (RectTransform)fieldGo.transform;
    rt.sizeDelta = new Vector2(600f, 64f);
    rt.anchoredPosition = new Vector2(0f, y);
    var bg = fieldGo.GetComponent<Image>();
    bg.color = new Color(0.16f, 0.14f, 0.20f, 0.95f);
    var field = fieldGo.GetComponent<InputField>();
    field.targetGraphic = bg; // U184: real target graphic for tint states

    var textGo = new GameObject("TextField", typeof(RectTransform), typeof(Text));
    textGo.transform.SetParent(fieldGo.transform, false);
    var textRt = (RectTransform)textGo.transform;
    textRt.anchorMin = new Vector2(0f, 0f);
    textRt.anchorMax = new Vector2(1f, 1f);
    textRt.sizeDelta = new Vector2(-24f, -14f);
    textRt.anchoredPosition = new Vector2(12f, 0f);
    var text = textGo.GetComponent<Text>();
    text.font = Resources.GetBuiltinResource<Font>("LegacyRuntime.ttf");
    text.fontSize = 22;
    text.alignment = TextAnchor.MiddleLeft;
    text.color = Color.white;
    text.supportRichText = false; // pasted codes are plain text, never markup
    text.raycastTarget = false;
    field.textComponent = text;

    var placeholderGo = new GameObject("Placeholder", typeof(RectTransform), typeof(Text));
    placeholderGo.transform.SetParent(fieldGo.transform, false);
    var placeholderRt = (RectTransform)placeholderGo.transform;
    placeholderRt.anchorMin = new Vector2(0f, 0f);
    placeholderRt.anchorMax = new Vector2(1f, 1f);
    placeholderRt.sizeDelta = new Vector2(-24f, -14f);
    placeholderRt.anchoredPosition = new Vector2(12f, 0f);
    var placeholder = placeholderGo.GetComponent<Text>();
    placeholder.font = Resources.GetBuiltinResource<Font>("LegacyRuntime.ttf");
    placeholder.fontSize = 20;
    placeholder.alignment = TextAnchor.MiddleLeft;
    placeholder.color = new Color(1f, 1f, 1f, 0.40f);
    placeholder.text = LocalizeService.Get("share.placeholder");
    placeholder.raycastTarget = false;
    field.placeholder = placeholder;
    return field;
}

static Button MakeButton(Transform parent, string name, string label,
    float x, float y, float w, float h)
{
    var go = new GameObject(name, typeof(RectTransform), typeof(Image), typeof(Button));
    go.transform.SetParent(parent, false);
    var rt = (RectTransform)go.transform;
    rt.sizeDelta = new Vector2(w, h);

```

```

        rt.anchoredPosition = new Vector2(x, y);
        var button = go.GetComponent<Button>();
        button.targetGraphic = go.GetComponent<Image>(); // U184: real target graphic
        button.transition = Selectable.Transition.ColorTint;
        var text = MakeText(go.transform, "Label", 16f, Vector2.zero, (int)w - 6, (int)h - 8);
        text.text = label;
        return button;
    }

    static Text MakeText(Transform parent, string name, float size, Vector2 pos, int w, int h)
    {
        var go = new GameObject(name, typeof(RectTransform), typeof(Text));
        go.transform.SetParent(parent, false);
        var rt = (RectTransform)go.transform;
        rt.sizeDelta = new Vector2(w, h);
        rt.anchoredPosition = pos;
        var t = go.GetComponent<Text>();
        t.font = Resources.GetBuiltinResource<Font>("LegacyRuntime.ttf");
        t.fontSize = Mathf.RoundToInt(size);
        t.alignment = TextAnchor.MiddleCenter;
        t.color = Color.white;
        t.raycastTarget = false; // never steal a button tap
        return t;
    }
}

// ===== FILE: Assets\_Game\Scripts\Runtime\UI\PageArt.cs =====
// T011-b page-art wiring helper: the three reskinned pages (S02 board,
// S03 codex, S04 share) build one art node each and apply the produced
// sprite through this seam so the gate can assert the null-safe law in
// one place. Art missing (catalog empty, art pruned, name typo) means the
// node sleeps and the procedural color-block face stays untouched - the
// same contract ArtRuntime.Get already enforces for every consumer.
using UnityEngine;
using UnityEngine.UI;

namespace Sy.BloomHaven.UI
{
    public static class PageArt
    {
        /// <summary>
        /// Build the page-art node (anchored center, behind every sibling)
        /// and apply the sprite. Returns the node whether or not art lives.
        /// </summary>
        public static GameObject BuildNode(Transform parent, string name,
            float x, float y, float w, float h, Sprite art)
        {
            var go = new GameObject(name, typeof(RectTransform), typeof(Image));
            go.transform.SetParent(parent, false);
            var rt = (RectTransform)go.transform;

```

```

        rt.sizeDelta = new Vector2(w, h);
        rt.anchoredPosition = new Vector2(x, y);
        rt.SetAsFirstSibling(); // page art sits behind every sibling face
        Apply(go.GetComponent<Image>(), art);
        return go;
    }

    /// <summary>
    /// Null-safe sprite apply. Returns true when the node shows live art;
    /// a null sprite deactivates the node so the procedural look stays.
    /// </summary>
    public static bool Apply(Image target, Sprite art)
    {
        if (target == null) return false;
        if (art == null)
        {
            target.sprite = null;
            target.color = Color.white;
            target.gameObject.SetActive(false);
            return false;
        }
        target.sprite = art;
        target.color = Color.white;
        target.preserveAspect = true;
        target.raycastTarget = false; // backdrop never steals a tap
        target.gameObject.SetActive(true);
        return true;
    }
}
}
}

```